Microsoft

Azure Cosmos DB

Data Modeling & Optimization

Session's objectives

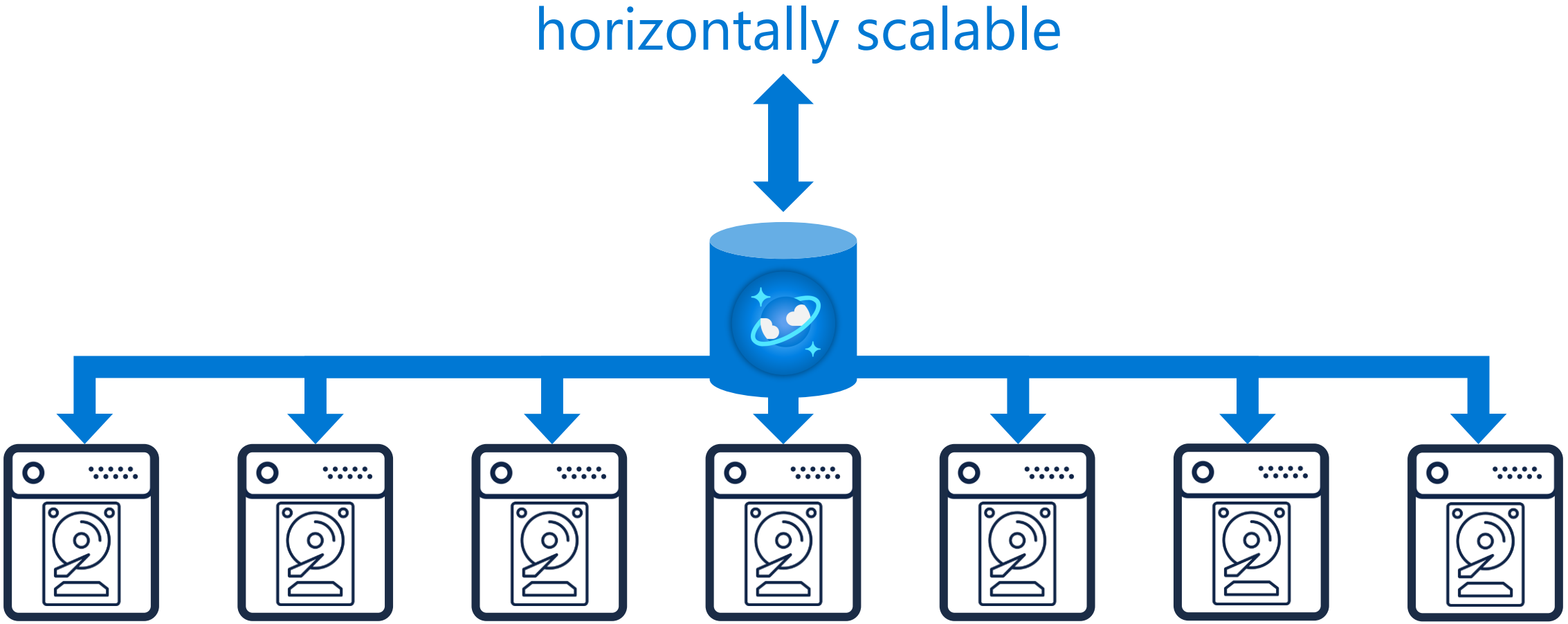
- Understand key data modeling concepts and best practices
- Understand key concepts for partitioning
- Apply them to a real-world scenario
- Understand how Cosmos DB differs from relational databases when designing a data model

What is Azure Cosmos DB?

Microsoft's ~~NoSQL~~ database on Azure

Non-relational and horizontally scalable

What is Azure Cosmos DB?



What is Azure Cosmos DB?

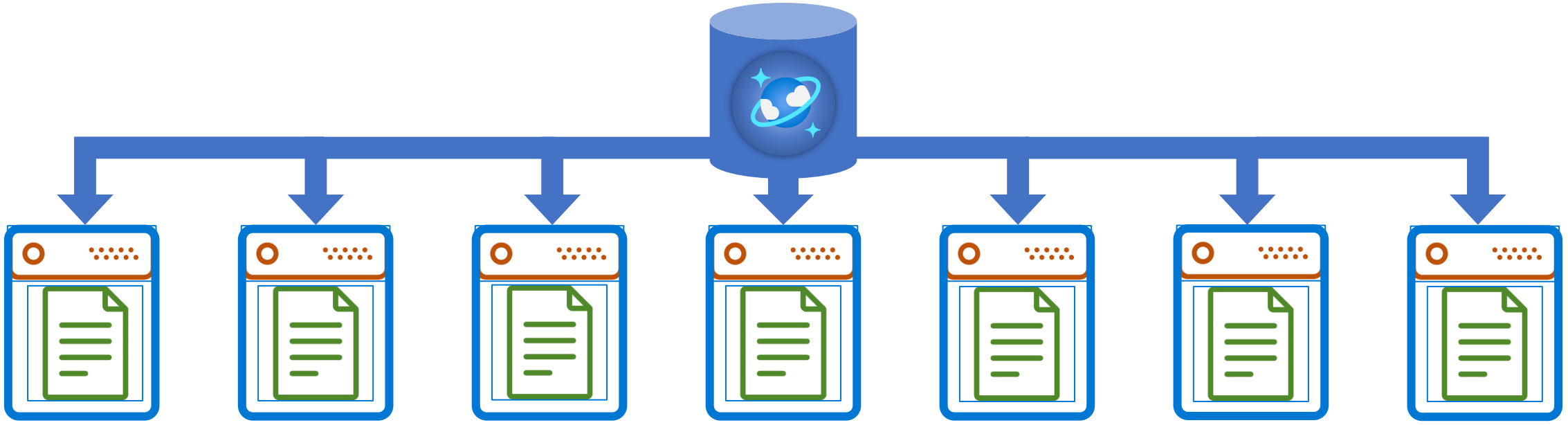
non-relational

What is Azure Cosmos DB?

non-relational

and

horizontally scalable



So is Cosmos DB suitable for relational workloads?

- It sure is!
  - Most workloads in Cosmos are actually relational
- Different techniques used to implement relationships
- Not intuitive. Relational best practices don't translate well and may even be anti-patterns!

Storage in 1970

- IBM 3330 data storage
- 200 MB to 1600 MB
- $74,000-$87,000 (1970)
- $567K - $666K (2022)

Comparing relational vs. NoSQL

- 1MB of storage in 1970 =~$2000-3000 [1][2]
- 1GB of storage in 1970 = $2,500,000
- 1GB of storage in Cosmos DB today = $.25c

- If I'm designing database storage in 1970 what am I optimizing for?

- Today compute, relative to storage is more expensive.
- NoSQL databases optimize for compute (requests) vs. storage.

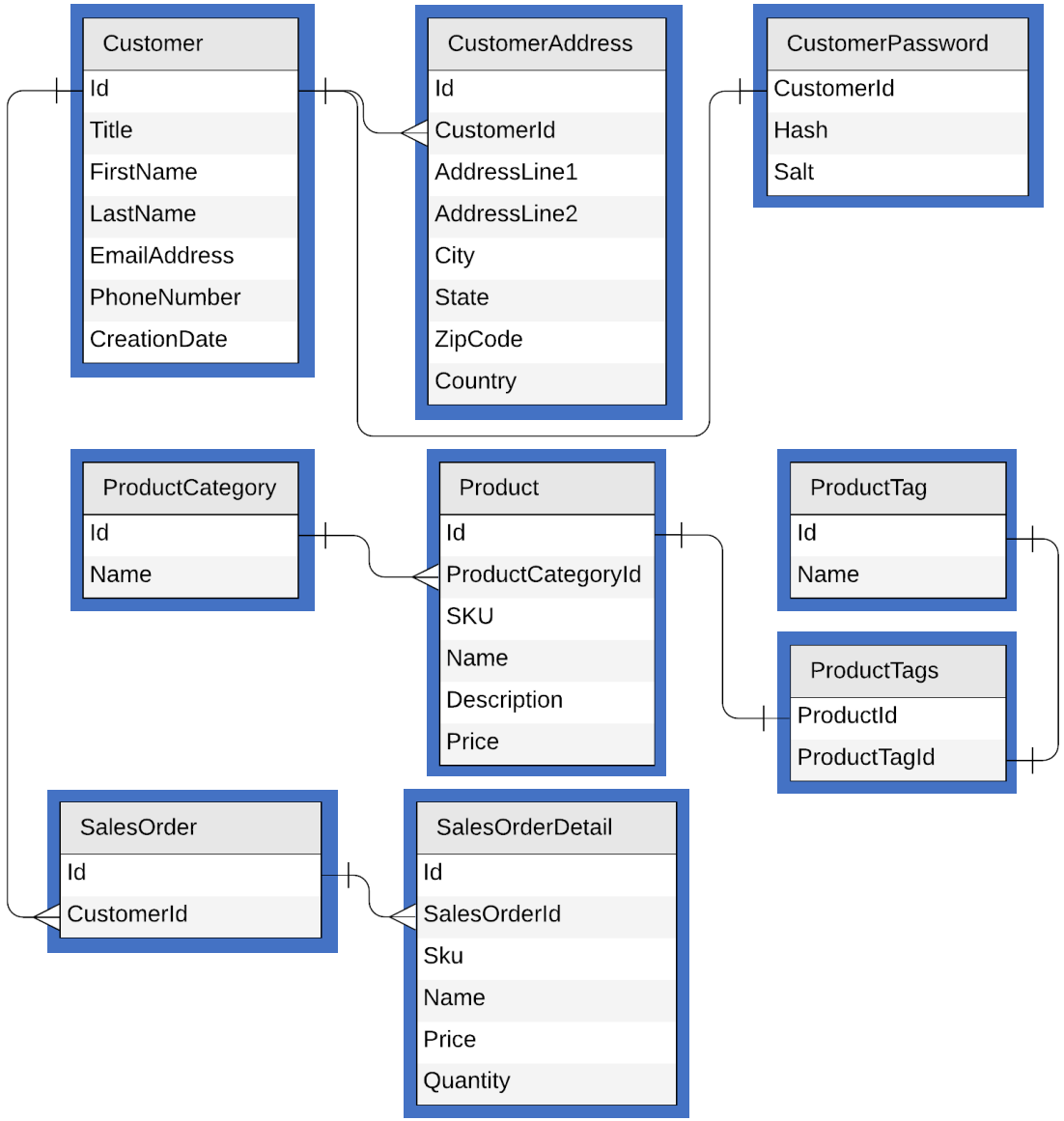
[1] - https://jcmit.net/diskprice.htm

[2] – Inflation rate 666.14% 1970 - 2022

Let's look at a concrete example

- E-commerce workload

- Customers
- Products
  - Categories
  - Tags
- Sales Orders



Identifying the operations for our app

- Create a customer
- Edit a customer
- Retrieve a customer

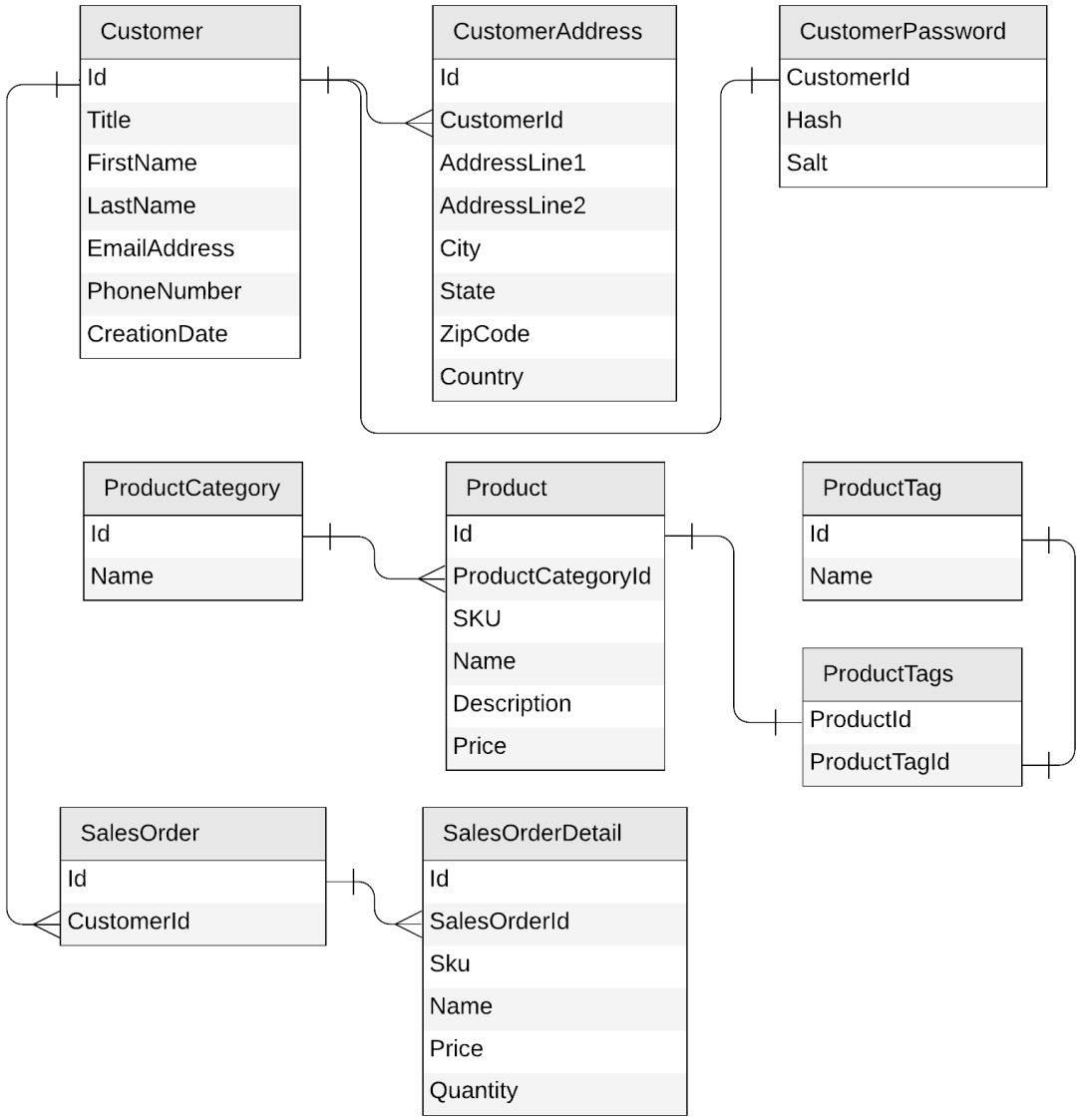
- Create a product category
- Edit a product category
- List all product categories

- Create a product tag
- Edit a product tag
- List all product tags

- Create a product
- Edit a product
- List all products from a category (with name of category and tags)

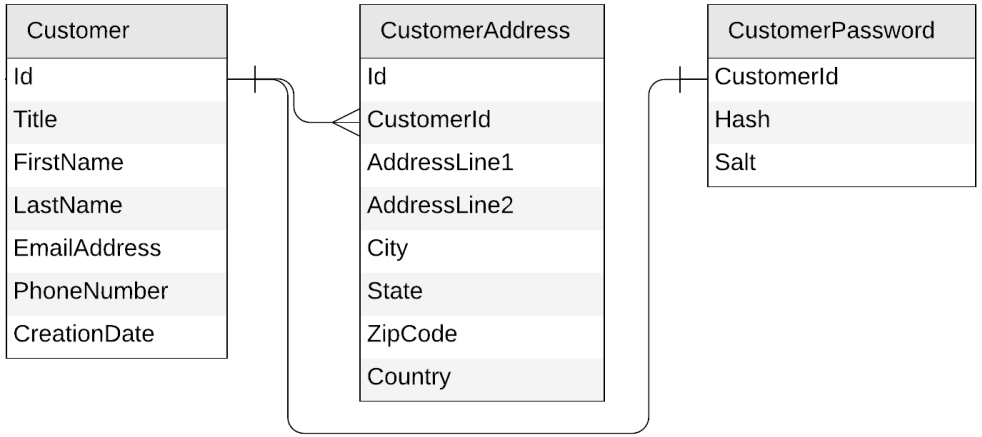
- Create a sales order
- List all sales order for a customer
- Query top 10 customers by number of sales orders

Let’s get started!

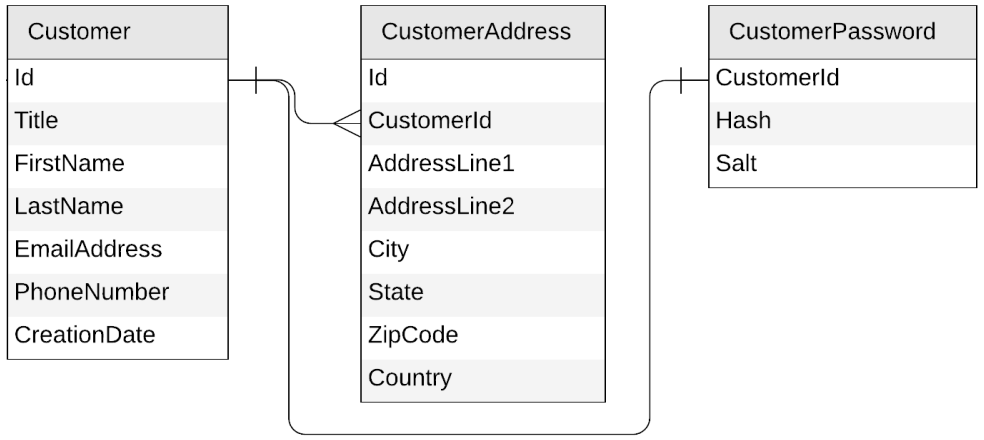


Starting with the Customer entity

- Create a customer
- Edit a customer
- Retrieve a customer



Starting with the Customer entity




```
{
  "id": "...",
  "title": "...",
  "firstName": "...",
  "lastName": "...",
  "emailAddress": "...",
  "phoneNumber": "...",
  "creationDate": "...",
  "addresses": [{
    "addressLine1": "...",
    "addressLine2": "...",
    "city": "...",
    "state": "...",
    "zipCode": "...",
    "country": "..."
  }],
  "password": {
    "hash": "...",
    "salt": "..."
  }
}
```

To embed or to reference?

```
{
  "id": "...",
  "title": "...",
  "firstName": "...",
  "lastName": "...",
  "emailAddress": "...",
  "phoneNumber": "...",
  "creationDate": "...",
  "addresses": [{
    "addressLine1": "...",
    "addressLine2": "...",
    "city": "...",
    "state": "...",
    "zipCode": "...",
    "country": "..."
  }],
  "password": {
    "hash": "...",
    "salt": "..."
  }
}
```

```
{
  "id": "...",
  "title": "...",
  "firstName": "...",
  "lastName": "...",
  "emailAddress": "...",
  "phoneNumber": "...",
  "creationDate": "..."
}
```

```
{
  "id": "...",
  "customerId": "...",
  "addressLine1": "...",
  "addressLine2": "...",
  "city": "...",
  "state": "...",
  "zipCode": "...",
  "country": "..."
}
```

```
{
  "id": "...",
  "customerId ": "...",
  "hash": "...",
  "salt": "..."
}
```

To embed or to reference?

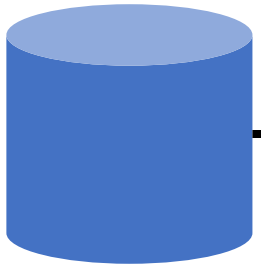
- Embed when:
  - 1:1 relationship
  - 1:few relationship
  - Related items are queried or updated together

- Reference when:
  - 1:many relationship (especially if unbounded)
  - Many:many relationship
  - Related items are queried or updated independently
    - though patch() can help with client side resources

Our first entity: Customer

```
{
  "id": "...",
  "title": "...",
  "firstName": "...",
  "lastName": "...",
  "emailAddress": "...",
  "phoneNumber": "...",
  "creationDate": "...",
  "addresses": [{
    "addressLine1": "...",
    "addressLine2": "...",
    ...
  }],
  "password": {
    "hash": "...",
    "salt": "..."
  }
}
```

Customer



customer
PK: ?


```
{
  "id": "...",
  "title": "...",
  "firstName": "...",
  "lastName": "...",
  "emailAddress": "...",
  "phoneNumber": "...",
  "creationDate": "...",
  "addresses": [{
    "addressLine1": "...",
    "addressLine2": "...",
    ...
  }],
  "password": {
    "hash": "...",
    "salt": "..."
  }
}
```

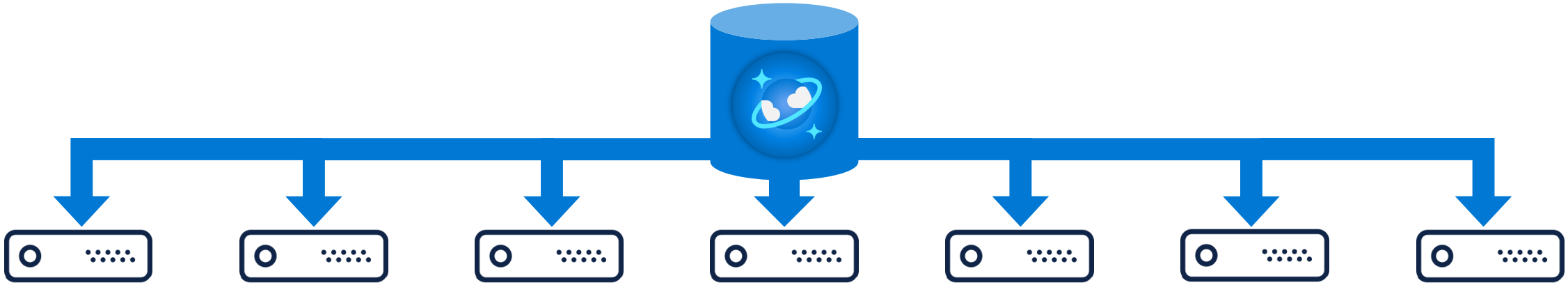
Key tips on document modeling

- Keep small but matched to access patterns
  - 1K=10RU
  - 256K=~100RU
  - 500K=~200RU
  - 1MB=~1000RU
- Shred documents with highly asymmetrical access patterns
- Flatter (3 max to keep optimal)
  - Cheaper & faster to index
  - Cheaper & faster to serialize & deserialize
- Indexing using “all except” pattern, so new properties are indexed
- Create a base document class when mixing entities and versioning

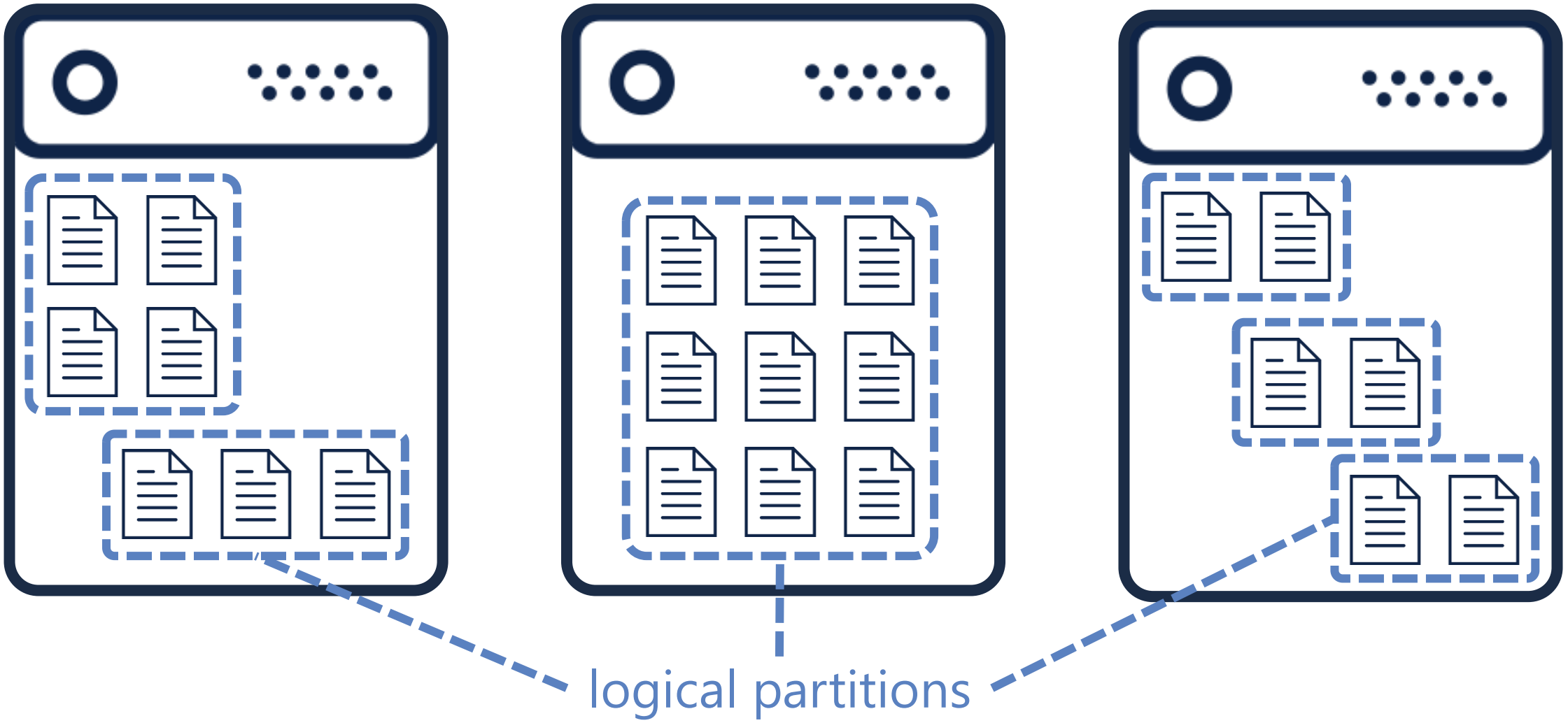
```
1 reference | 0 changes | 0 authors, 0 changes
public class documentBase
{
    0 references | 0 changes | 0 authors, 0 changes
    public string id { get; set; }
    0 references | 0 changes | 0 authors, 0 changes
    public string docType { get;set; }
    0 references | 0 changes | 0 authors, 0 changes
    public int docVersion { get; set; }
}

0 references | 0 changes | 0 authors, 0 changes
public class CustomerV5: documentBase
{
    0 references | 0 changes | 0 authors, 0 changes
    public string customerId { get; set; }
    0 references | 0 changes | 0 authors, 0 changes
    public string title { get; set; }
    0 references | 0 changes | 0 authors, 0 changes
    public string firstName { get; set; }
    0 references | 0 changes | 0 authors, 0 changes
    public string lastName { get; set; }
    0 references | 0 changes | 0 authors, 0 changes
    public string emailAddress { get; set; }
```

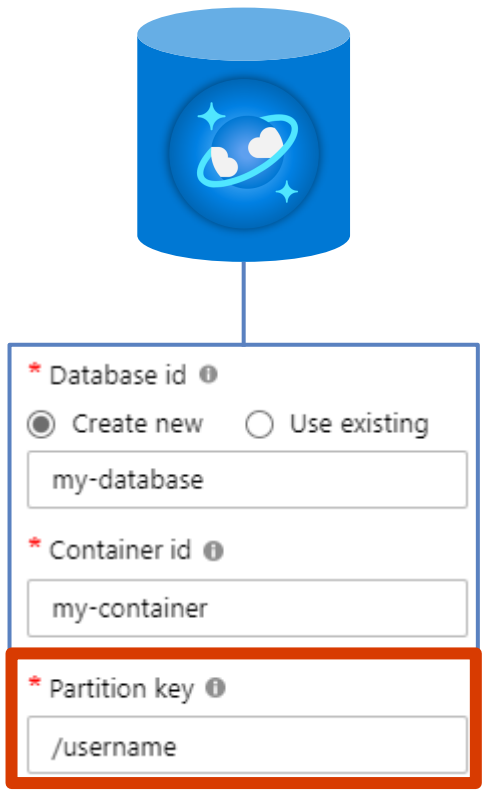
What is partitioning?

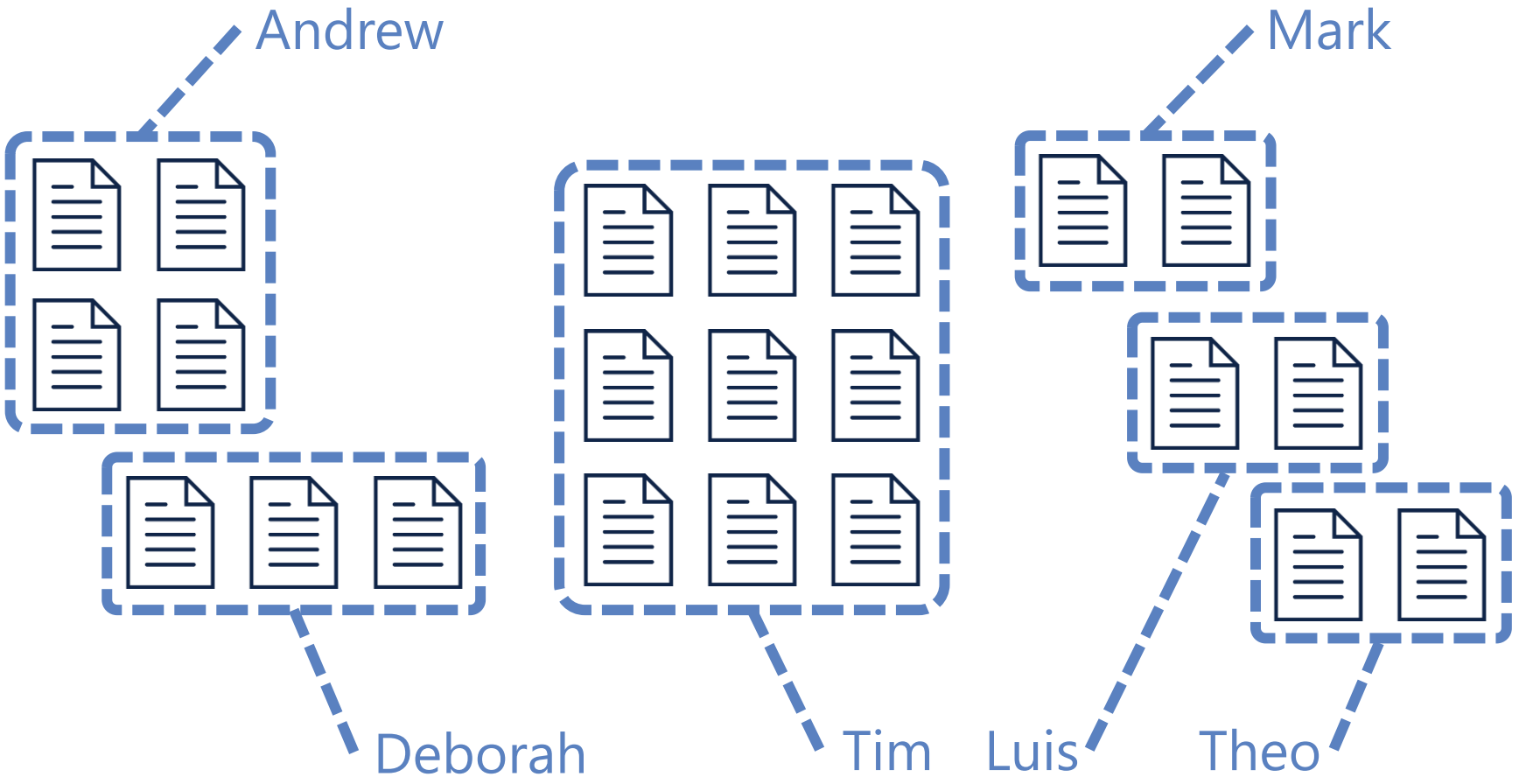


What is partitioning?

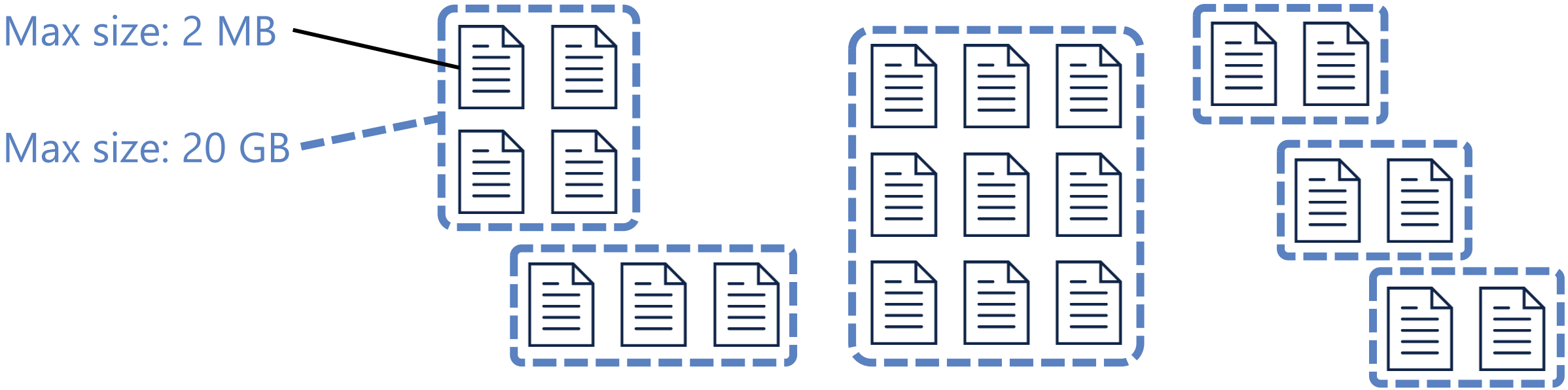


What is partitioning?

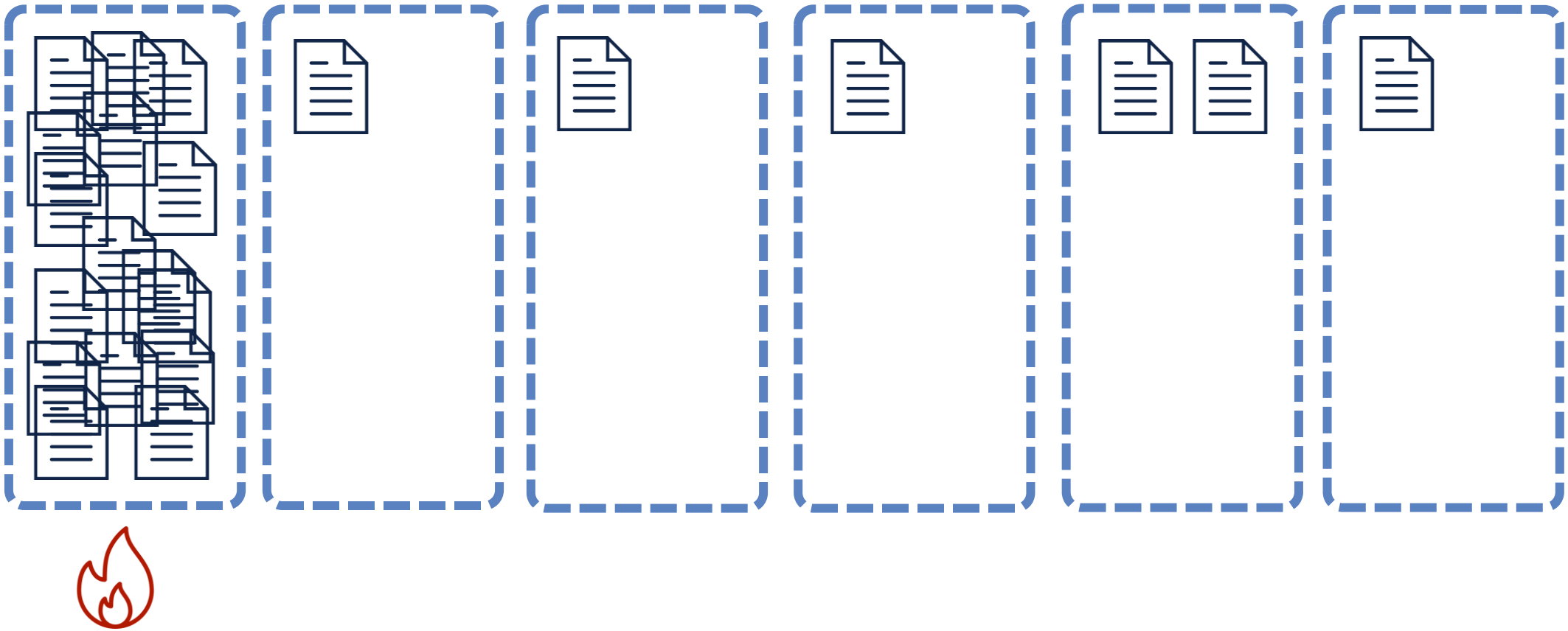




What is partitioning?



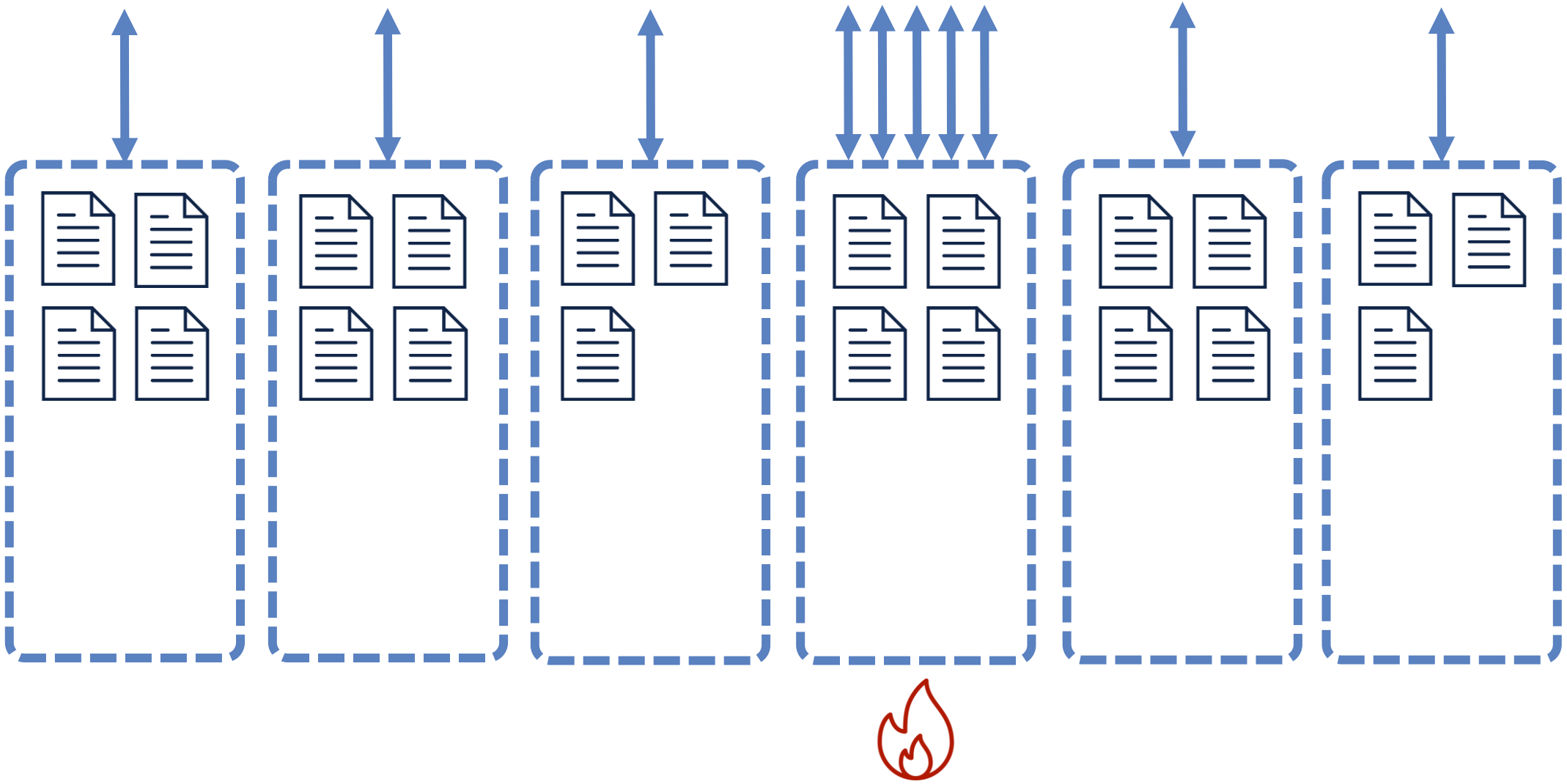
What is partitioning?



Hierarchical partition keys

https://aka.ms/cosmos-hierarchical-partitioning

What is partitioning?



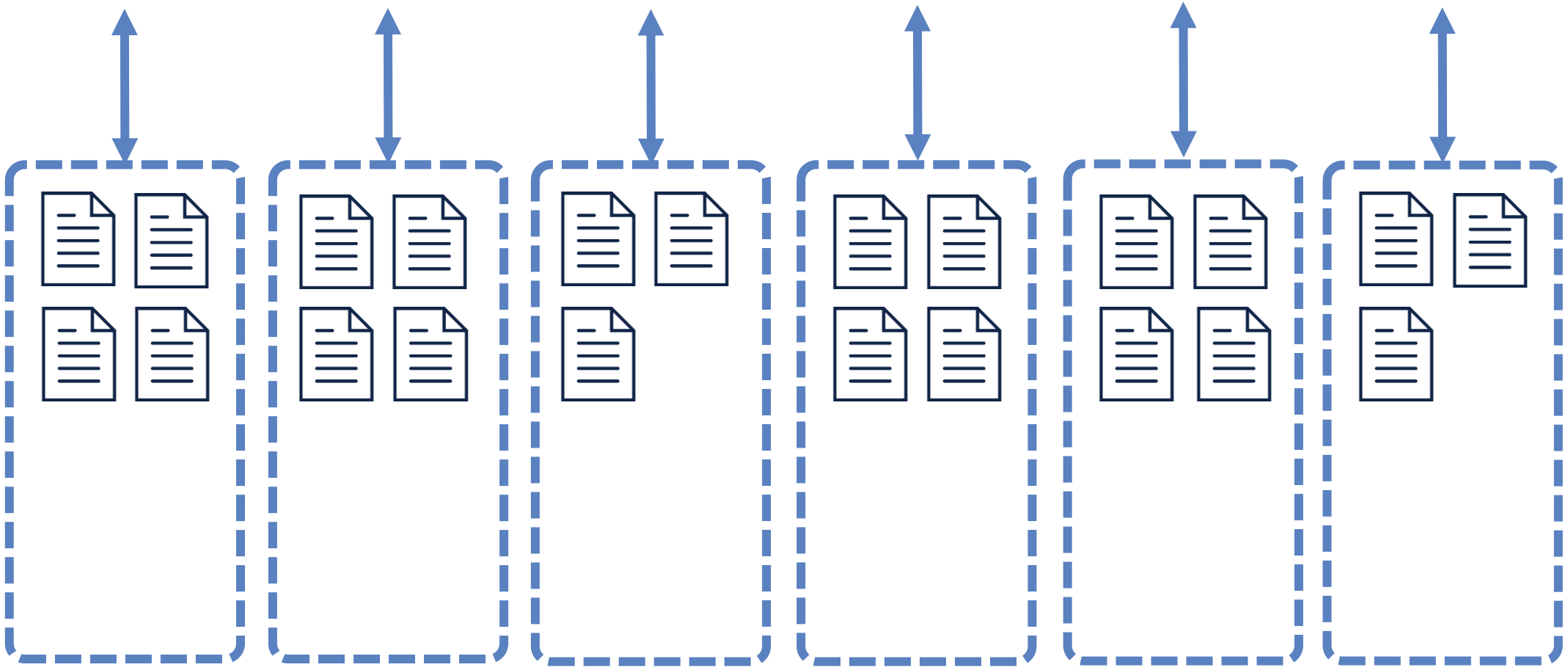
Redistribute throughput (preview)

https://aka.ms/CosmosRedistributeThroughput

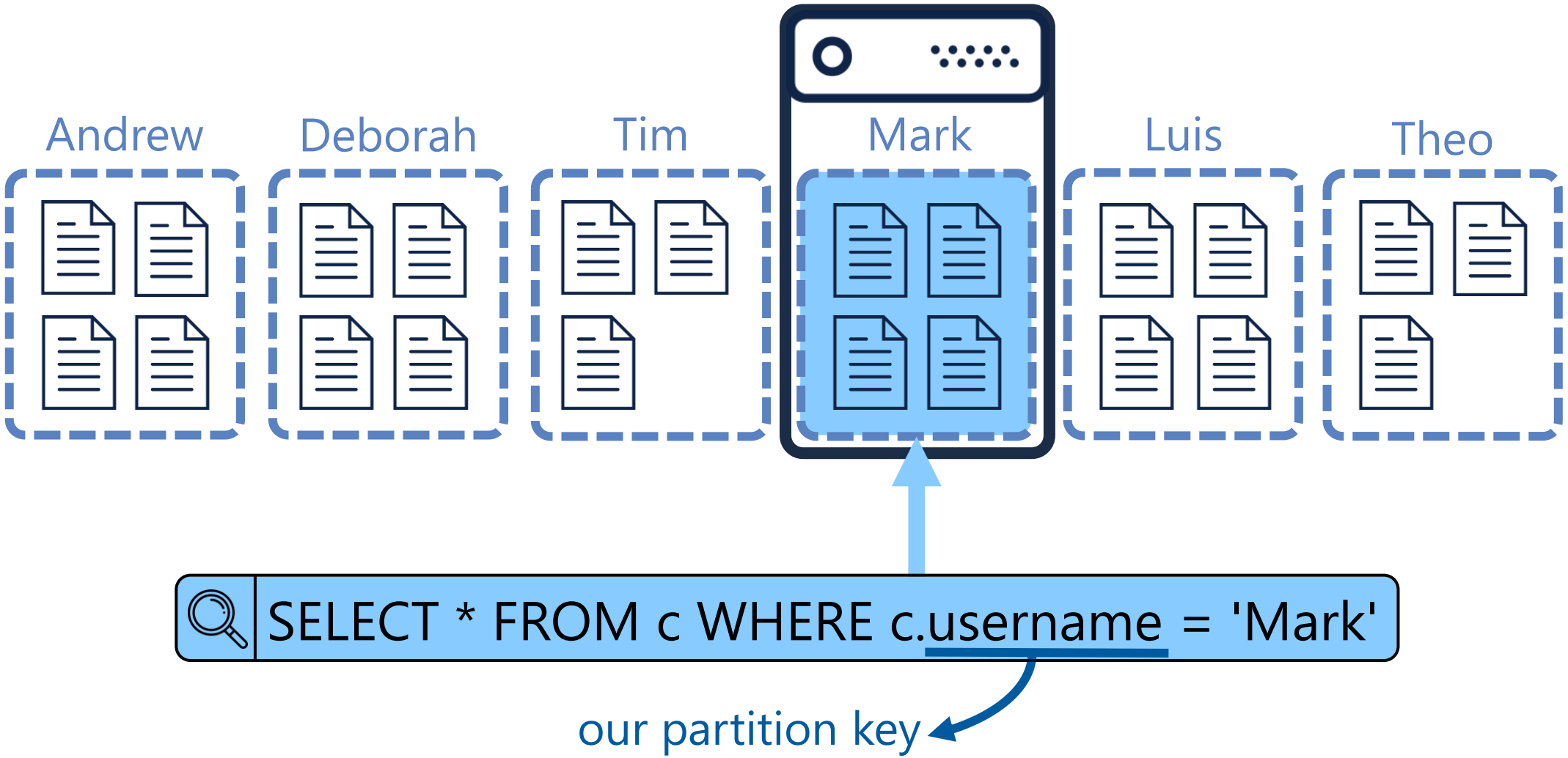
Dynamic Scaling

https://aka.ms/CosmosDynamicAutoscale

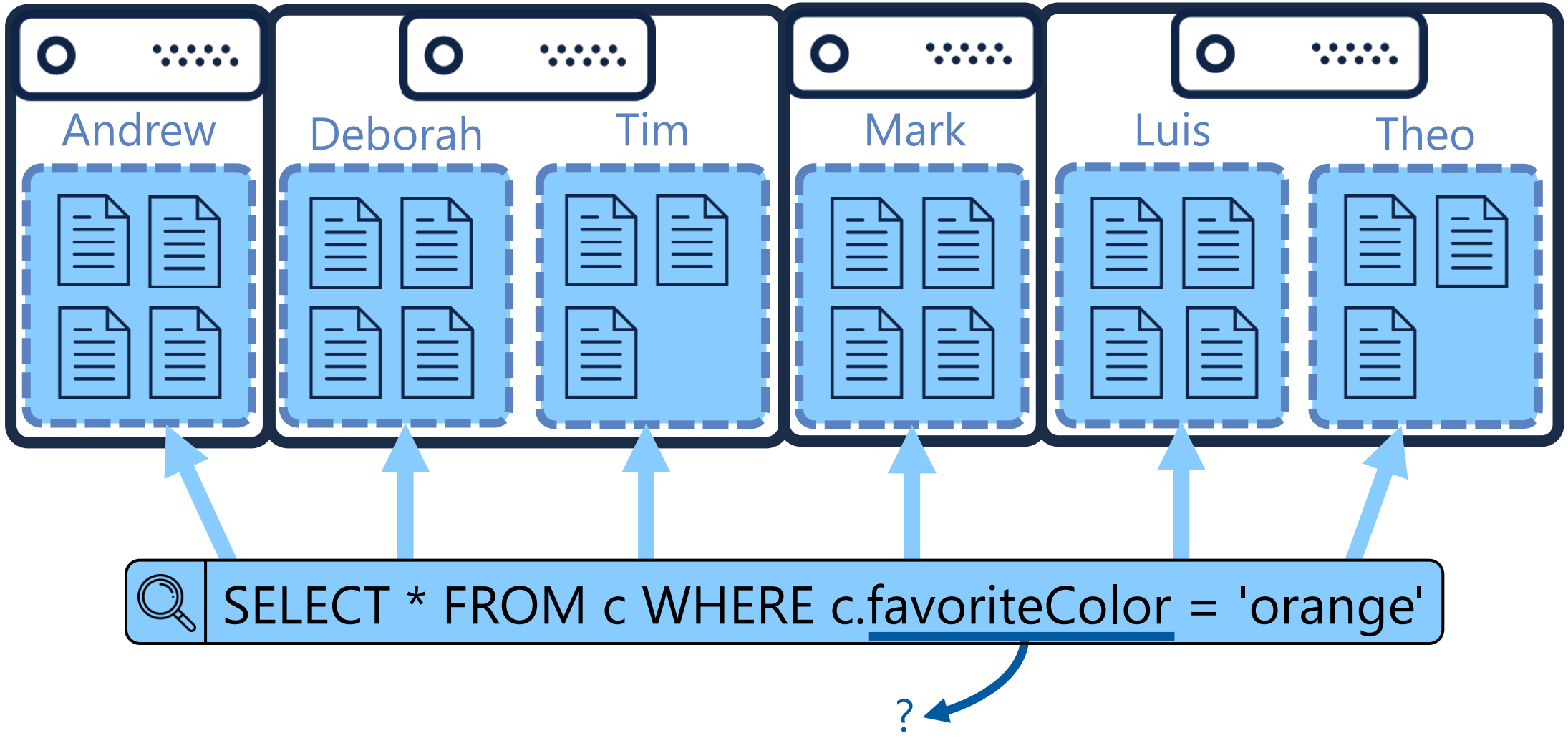
What is partitioning?



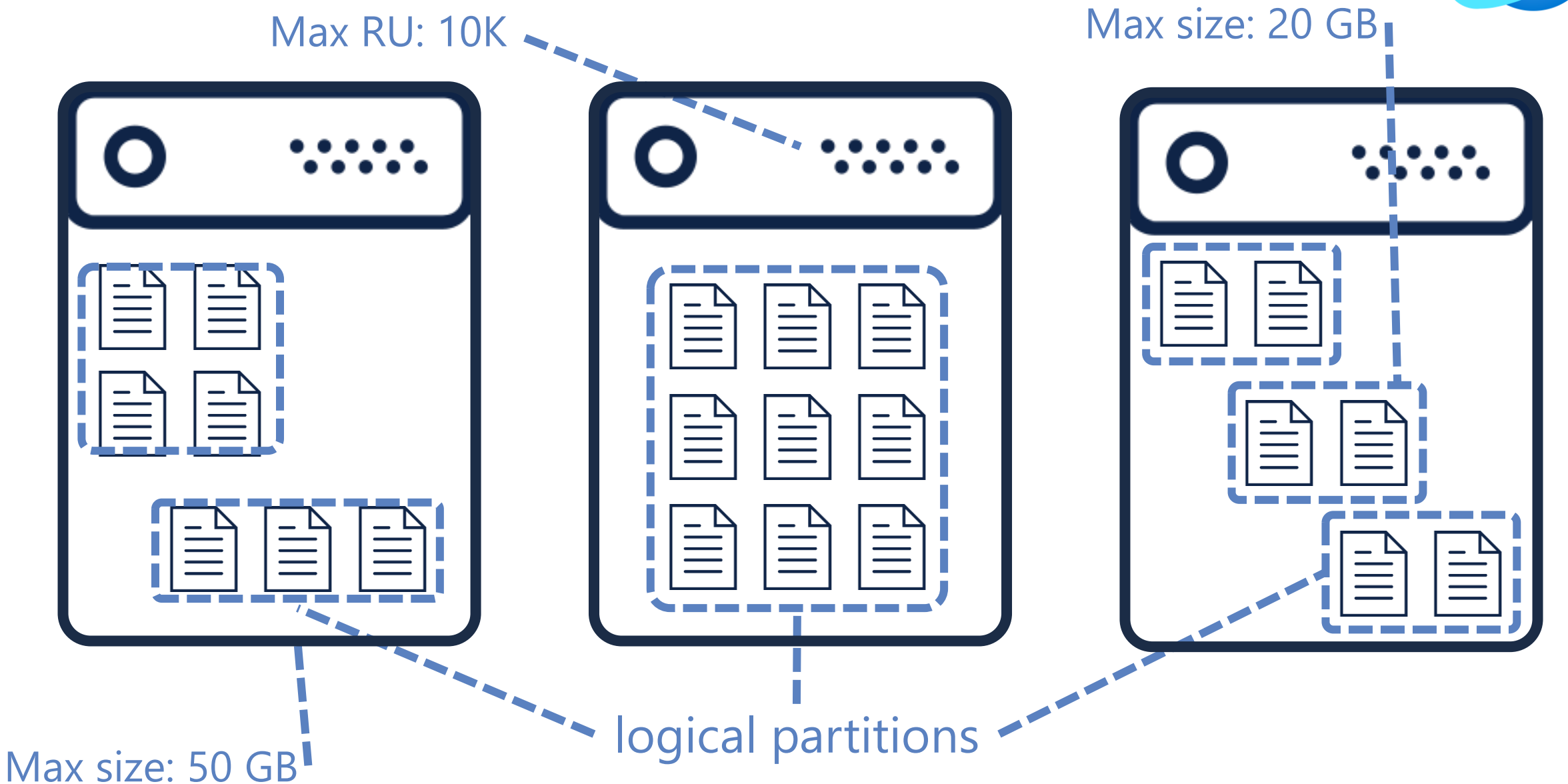
What is partitioning?



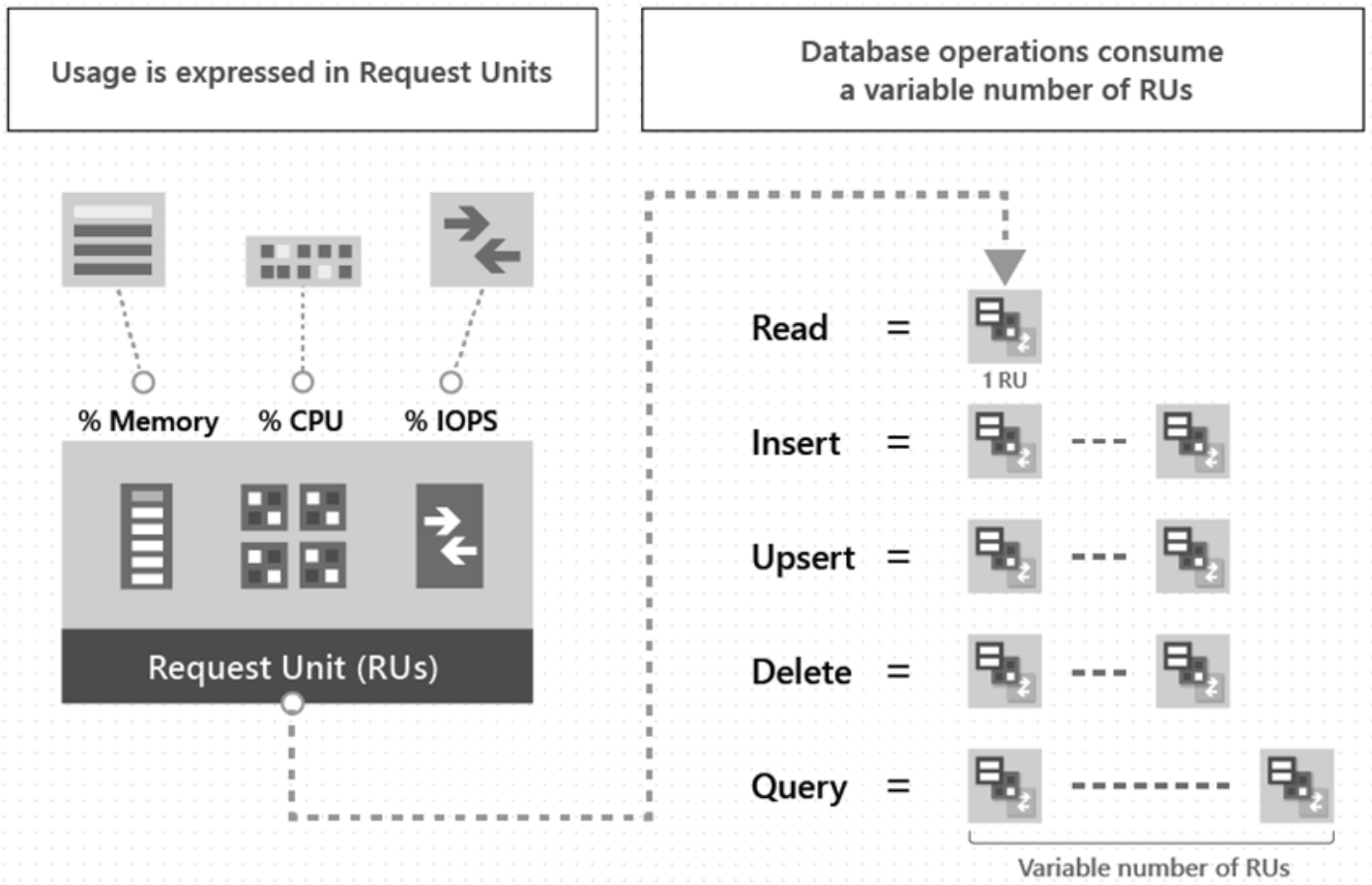
What is partitioning?



Physical vs logical partitions



What is an RU?



- Per SECOND!
- Amortize over time

- Monthly batch job
- 100,000 documents (~1kb)
- 10 RU/s insert
- 1M RU total

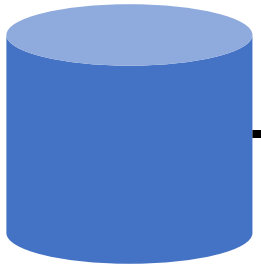
- Cost to insert
- 1 second?
- 10 seconds?
- 1 minute?
- 1 month?

learn.microsoft.com/azure/cosmos-db/request-units

Key tips on Throughput

- Throughput and partitions are bound to each other
- Heuristic for partition splits is more aggressive at container creation than afterwards.
- Massive scale-up then down again can result in throughput fragmentation and 429's later
  - 100,000 RU = 10 partitions
  - If I go from 100,000 RU -> 1,000 RU then…
  - 1000 RU/10 = 100 RU/partition (too little compute to do much)
- Read this before doing any major scale up
- learn.microsoft.com/azure/cosmos-db/scaling-provisioned-throughput-best-practices

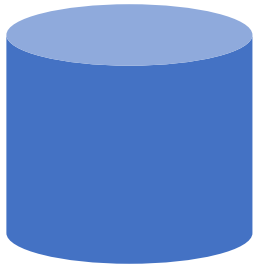
Choosing a partition key for customers



customer
PK: ?


```
{
  "id": "...",
  "title": "...",
  "firstName": "...",
  "lastName": "...",
  "emailAddress": "...",
  "phoneNumber": "...",
  "creationDate": "...",
  "addresses": [{
    "addressLine1": "...",
    "addressLine2": "...",
    ...
  }],
  "password": {
    "hash": "...",
    "salt": "..."
  }
}
```

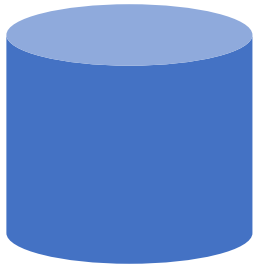
Choosing a partition key for customers



customer
PK: ?

- Create a customer
- Edit a customer
- Retrieve a customer (by id)

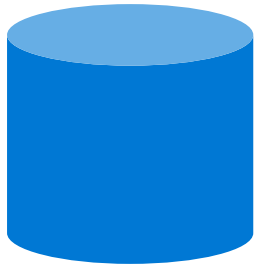
Choosing a partition key for customers



customer
PK: id

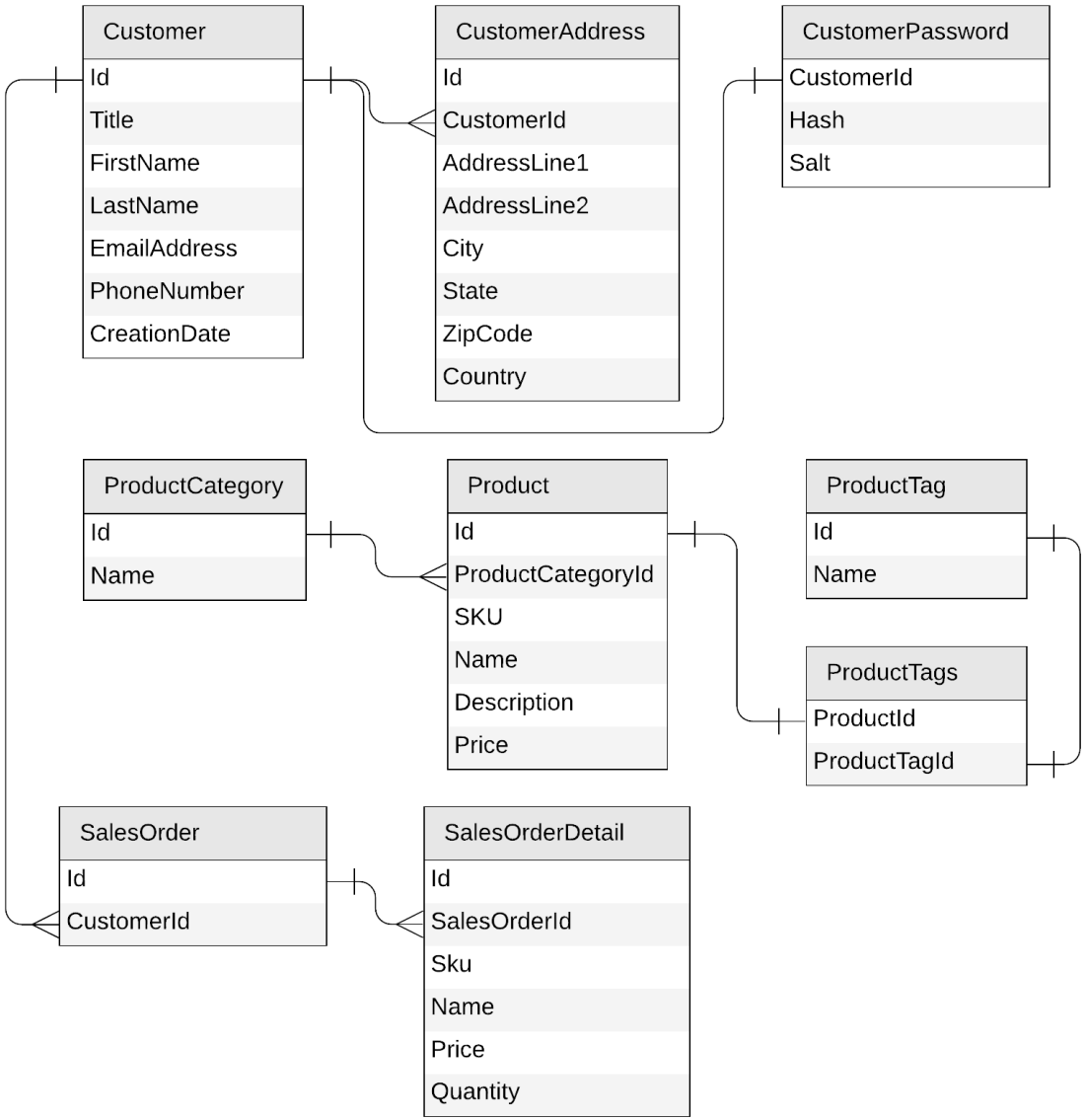
- Create a customer
- Edit a customer
- Retrieve a customer (by id)

Choosing a partition key for customers

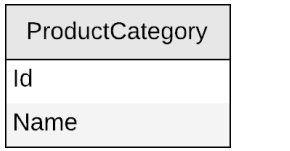


customers
PK: id

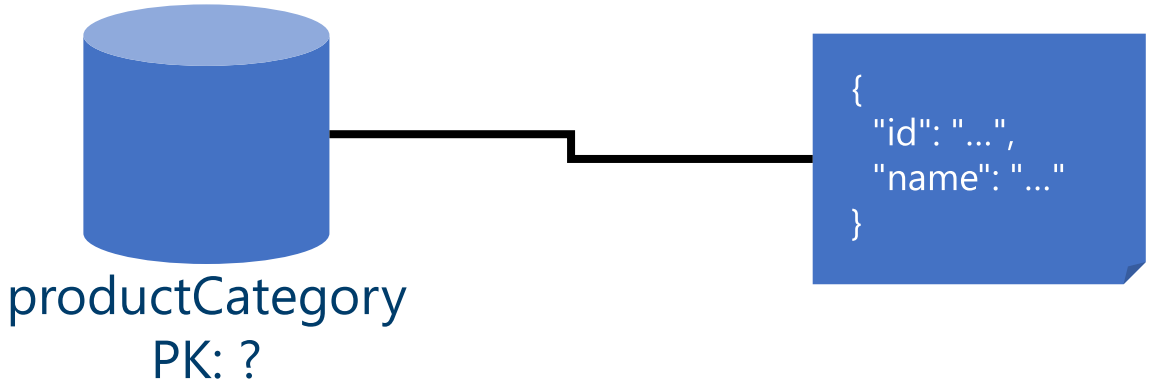
Next: products



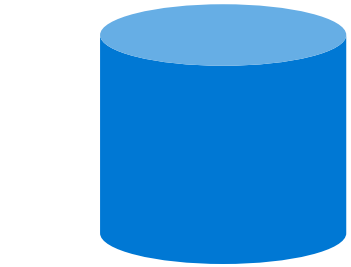
Product category



Product category



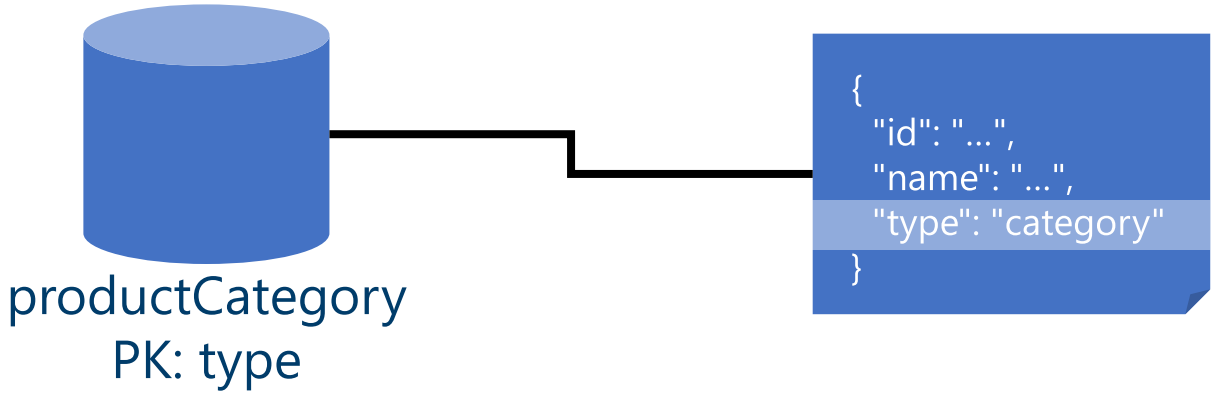
Product categories



productCategories
PK: ?

- Create a product category
- Edit a product category
- List all product categories

Product category

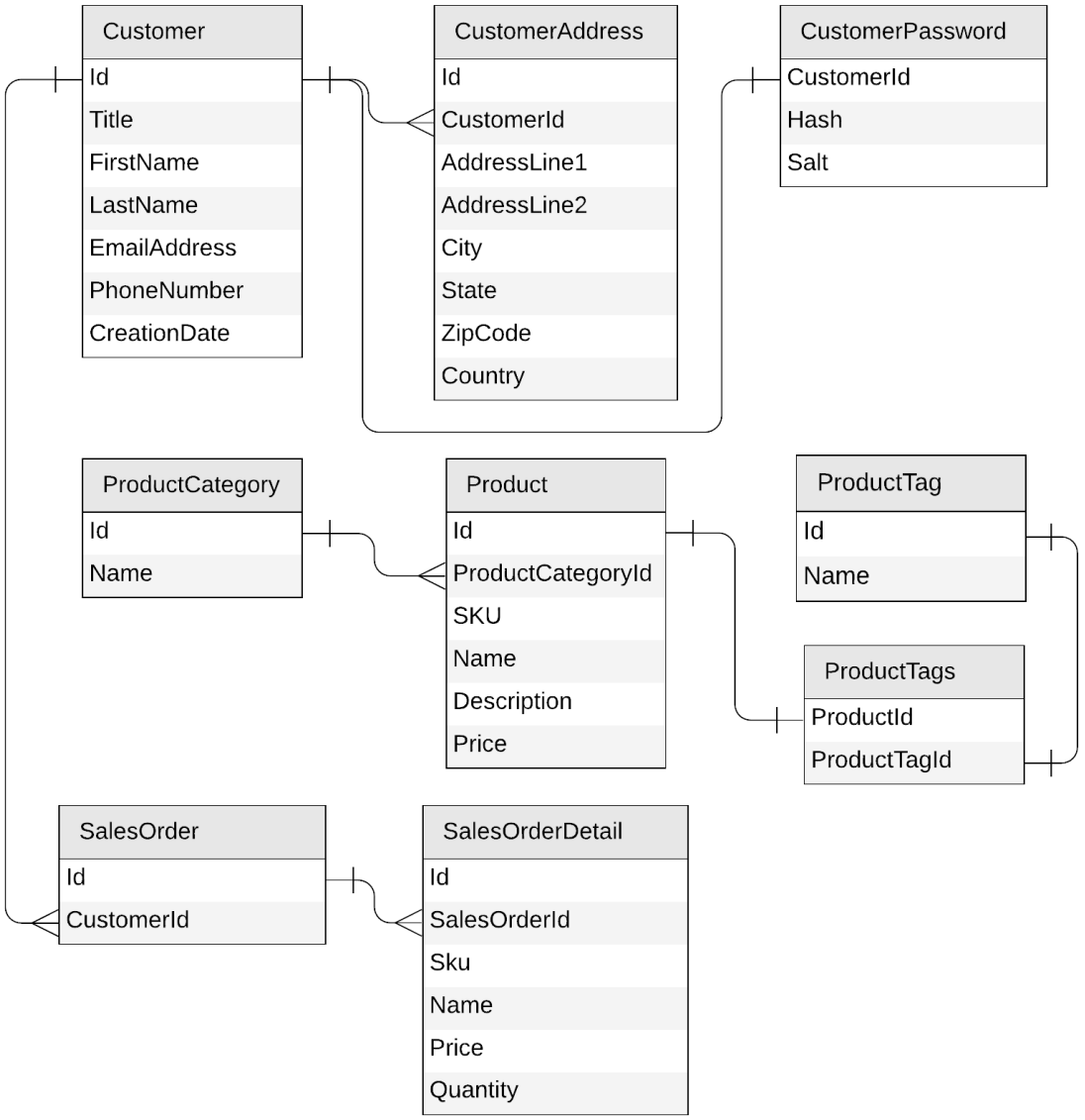


SELECT * FROM c WHERE
c.type = “category”

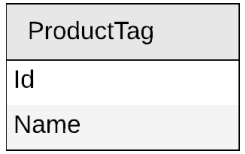
DEMO

Query product categories

Next: product tag

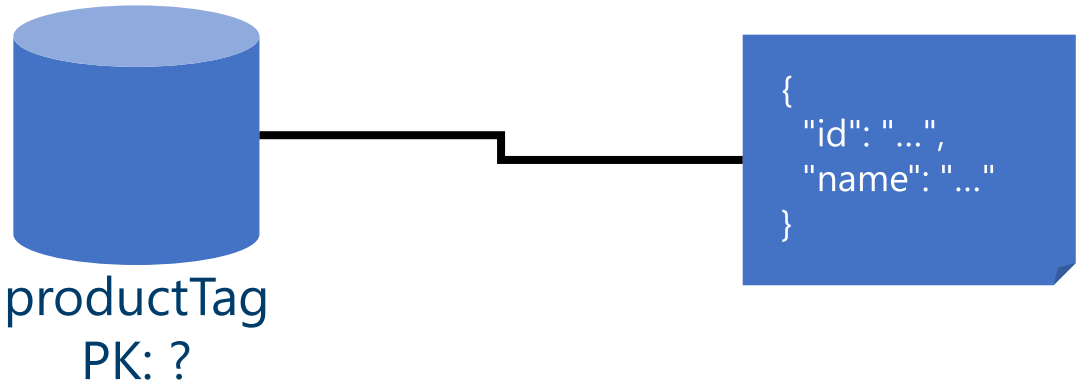


Product tag

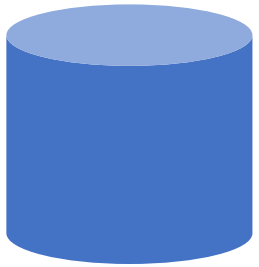



```
{
  "id": "...",
  "name": "..."
}
```

Product tag

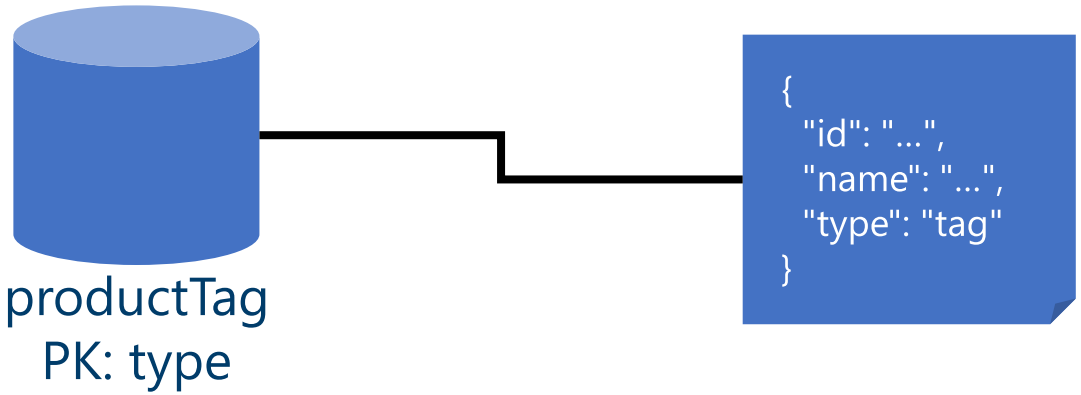


Product tag

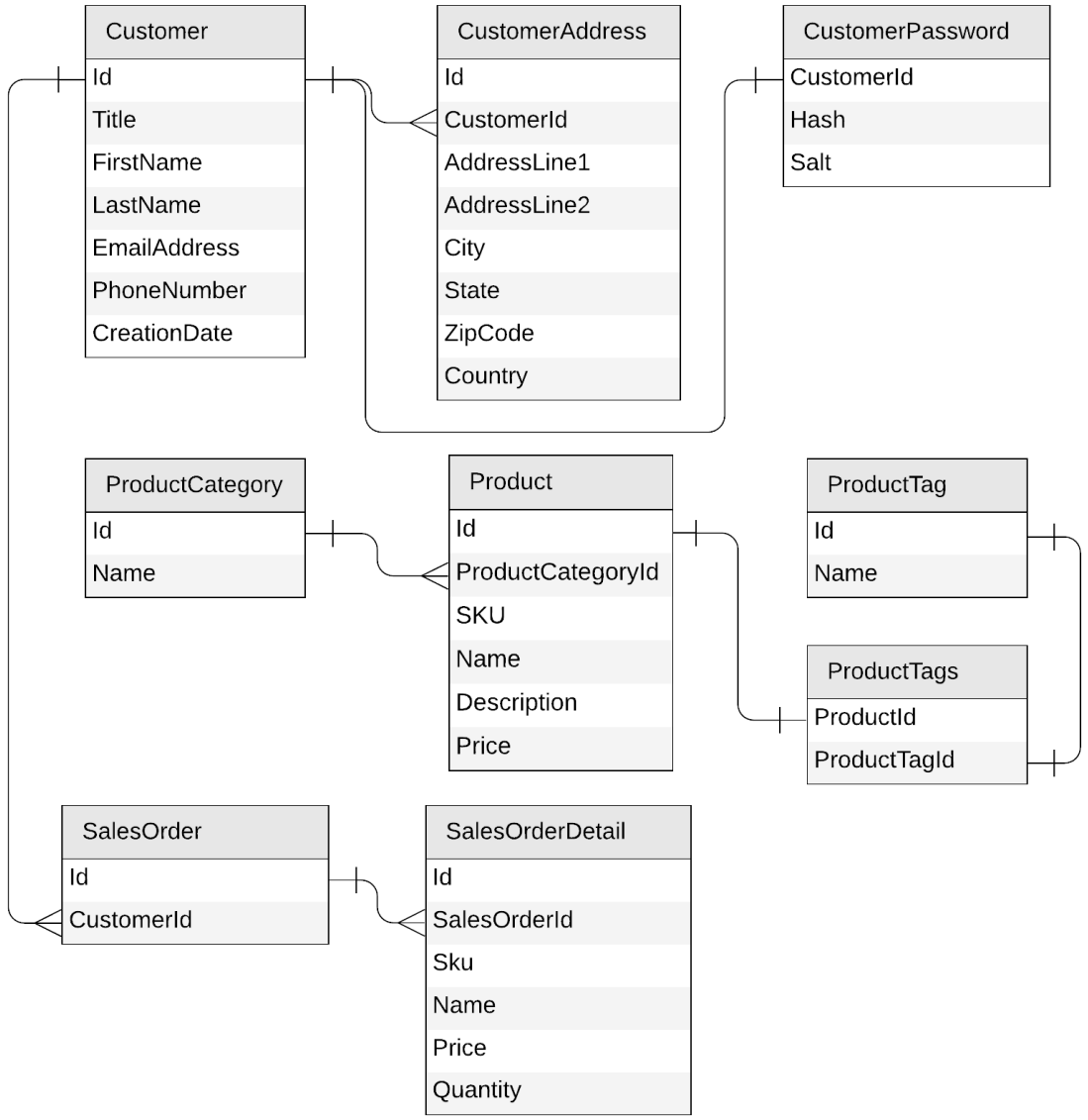


- Create a product tag
- Edit a product tag
- List all product tags

Product tag

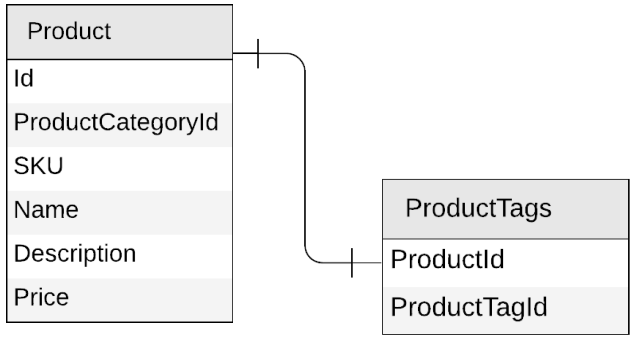


Next: product



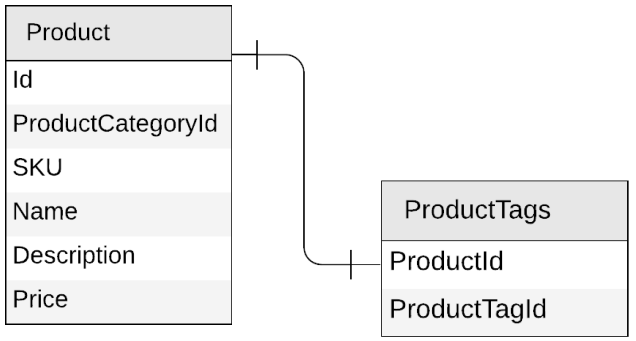
Product

```
{
  "id": "...",
  "categoryId": "...",
  "sku": "...",
  "name": "...",
  "description": "...",
  "price": ...
}
```

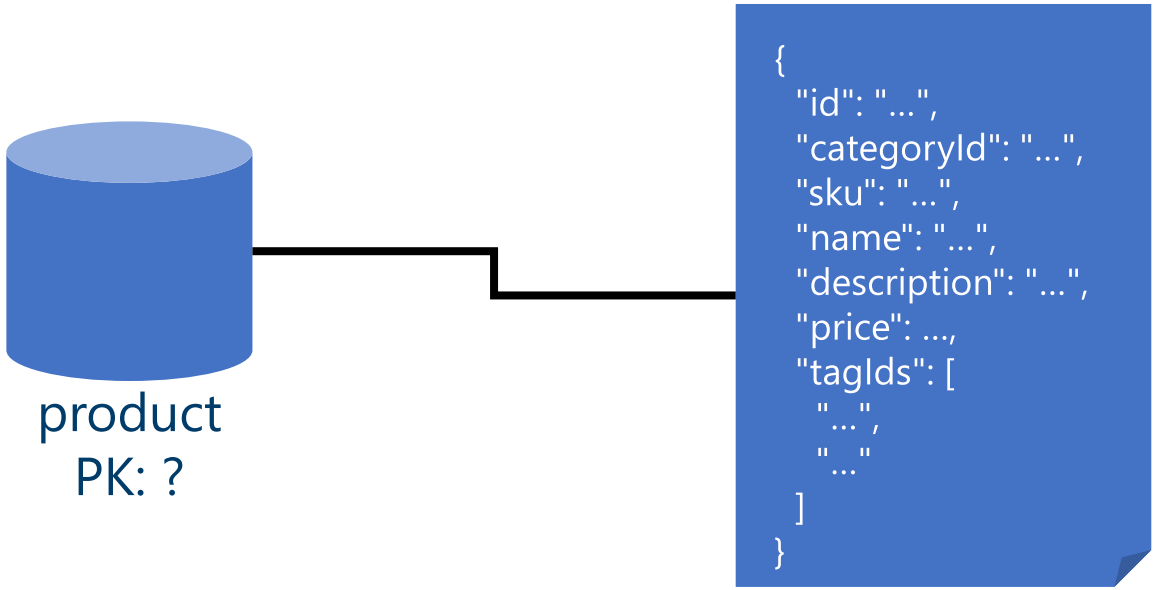


Products

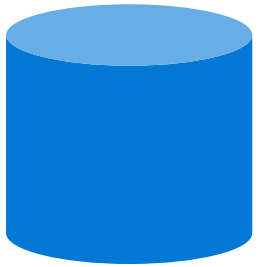

{
"id": "...",
"categoryId": "...",
"sku": "...",
"name": "...",
"description": "...",
"price": ...,
"tagIds": [
"...",
"..."
]
}


Product
Id
ProductCategoryId
SKU
Name
Description
Price
ProductTags
ProductId
ProductTagId

Products


product
PK: ?
{
"id": "...",
"categoryId": "...",
"sku": "...",
"name": "...",
"description": "...",
"price": ...,
"tagIds": [
"...",
"..."
]
}

Products



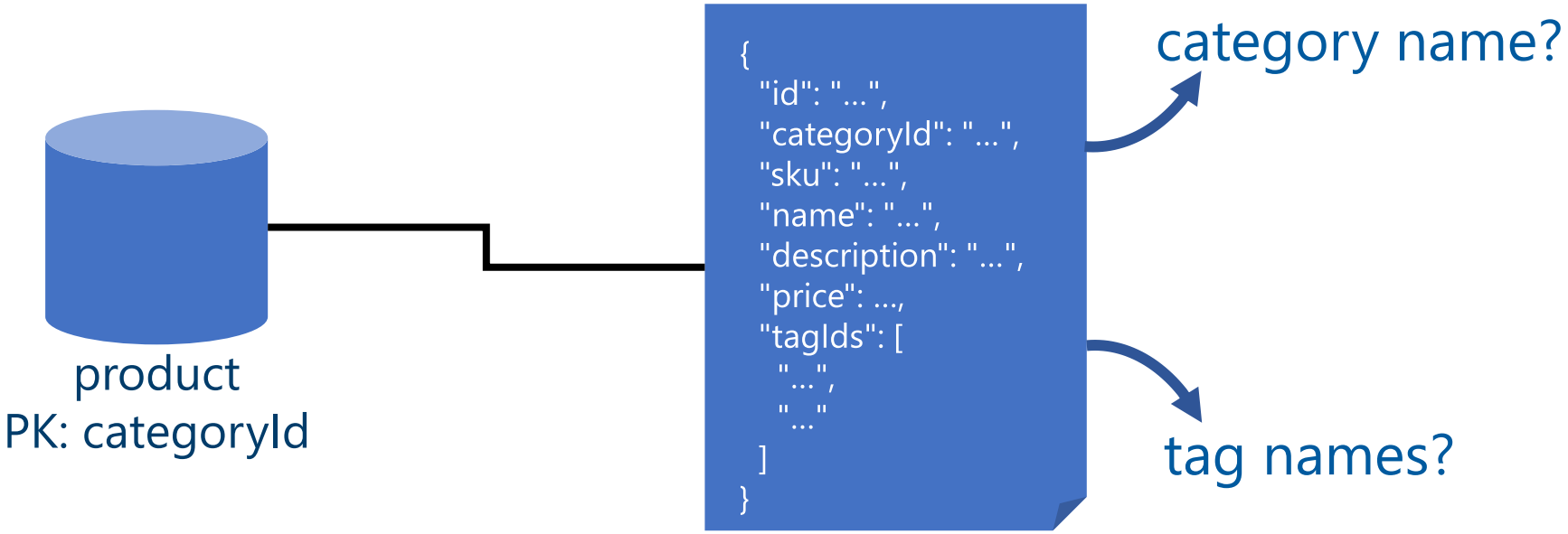
products
PK: ?

- Create a product
- Edit a product
- List all products from a category (with name of category and tags)

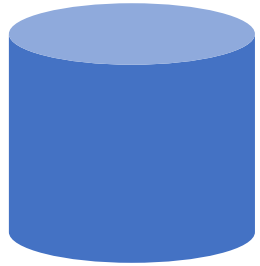
```
SELECT * FROM c WHERE c.categoryId = 'CategoryA'
```

CategoryA CategoryB CategoryC

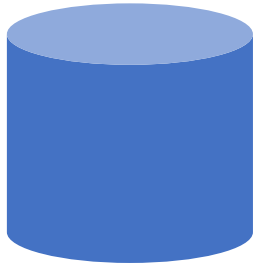
Products


{
"id": "...",
"categoryId": "...",
"sku": "...",
"name": "...",
"description": "...",
"price": ...,
"tagIds": [
"...",
"..."
]
}
category name?
tag names?
product
PK: categoryId

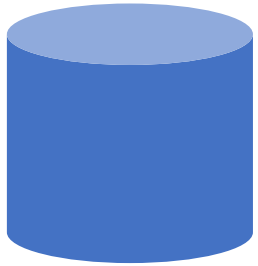
Products: how to return category and tag names?



product

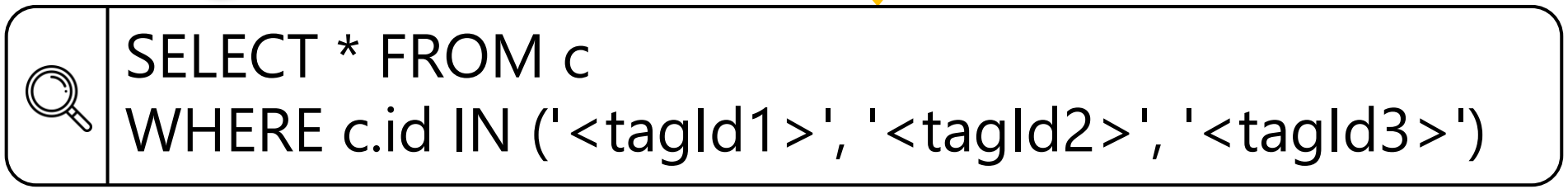


productCategory



productTag

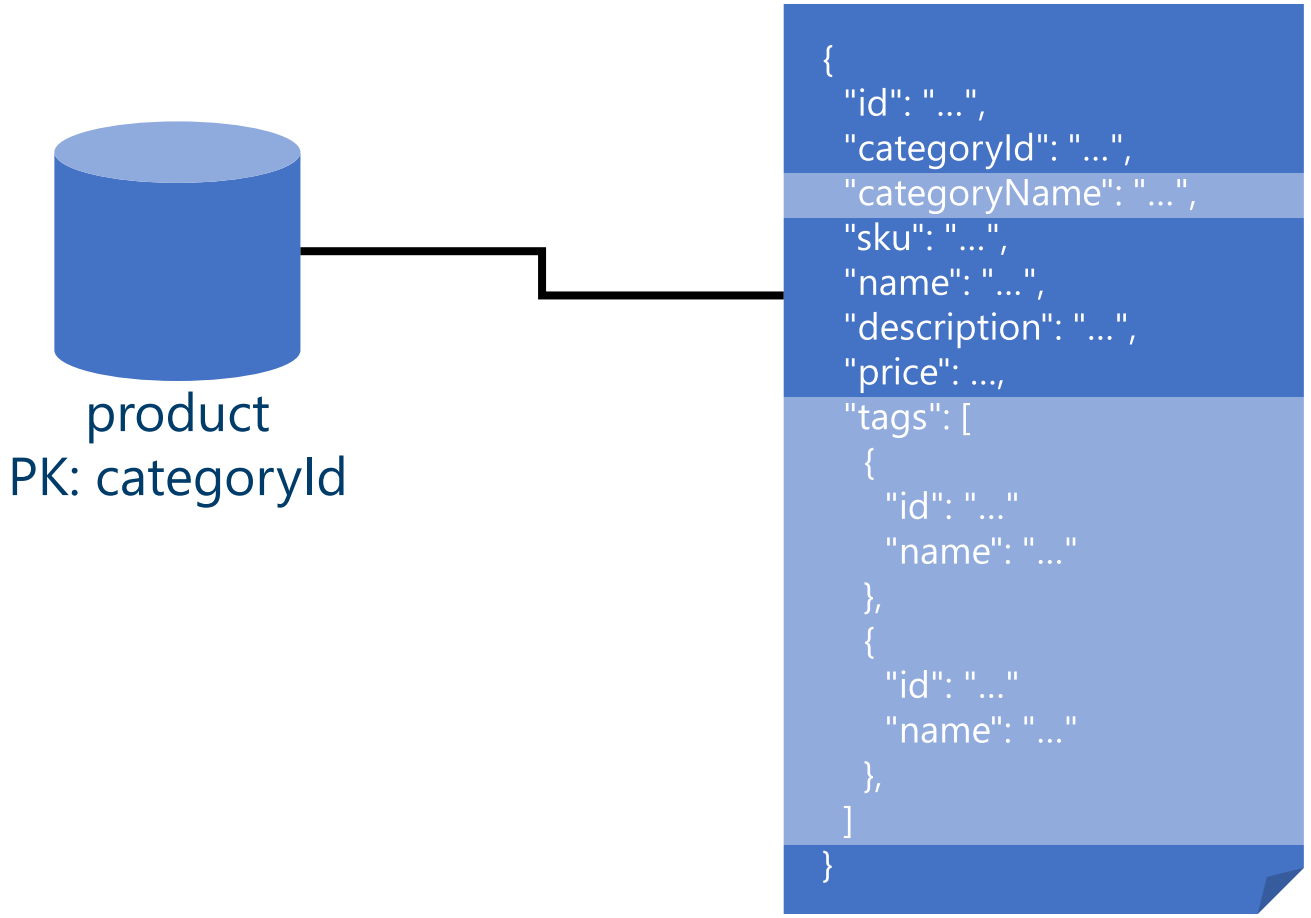
WHERE ARE MY JOINS?



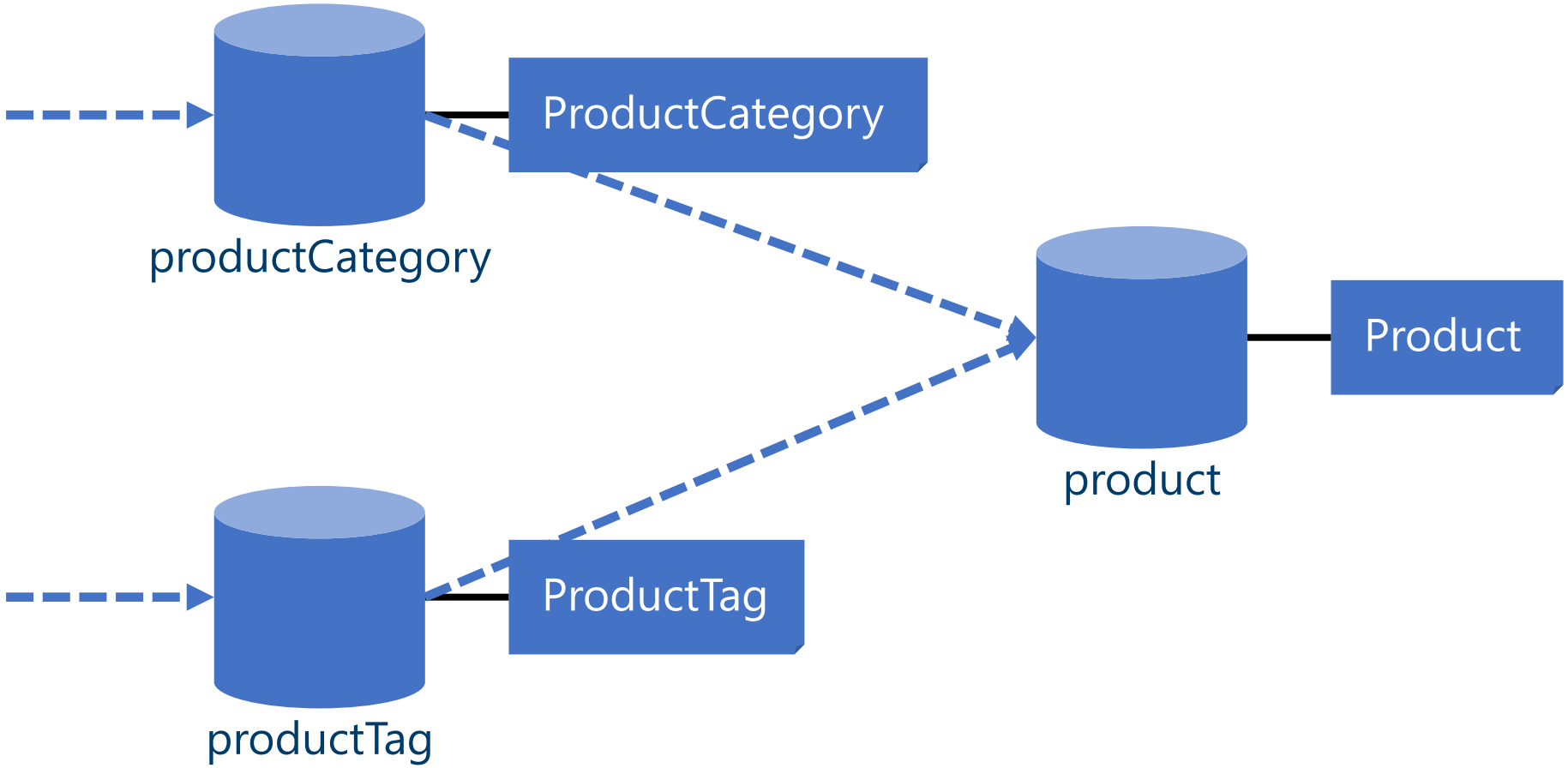
Introducing denormalization

- When working with non-relational databases, we **optimize our data models** to make sure that all the required data is **ready to be served** by the queries

Products: denormalizing category and tag names



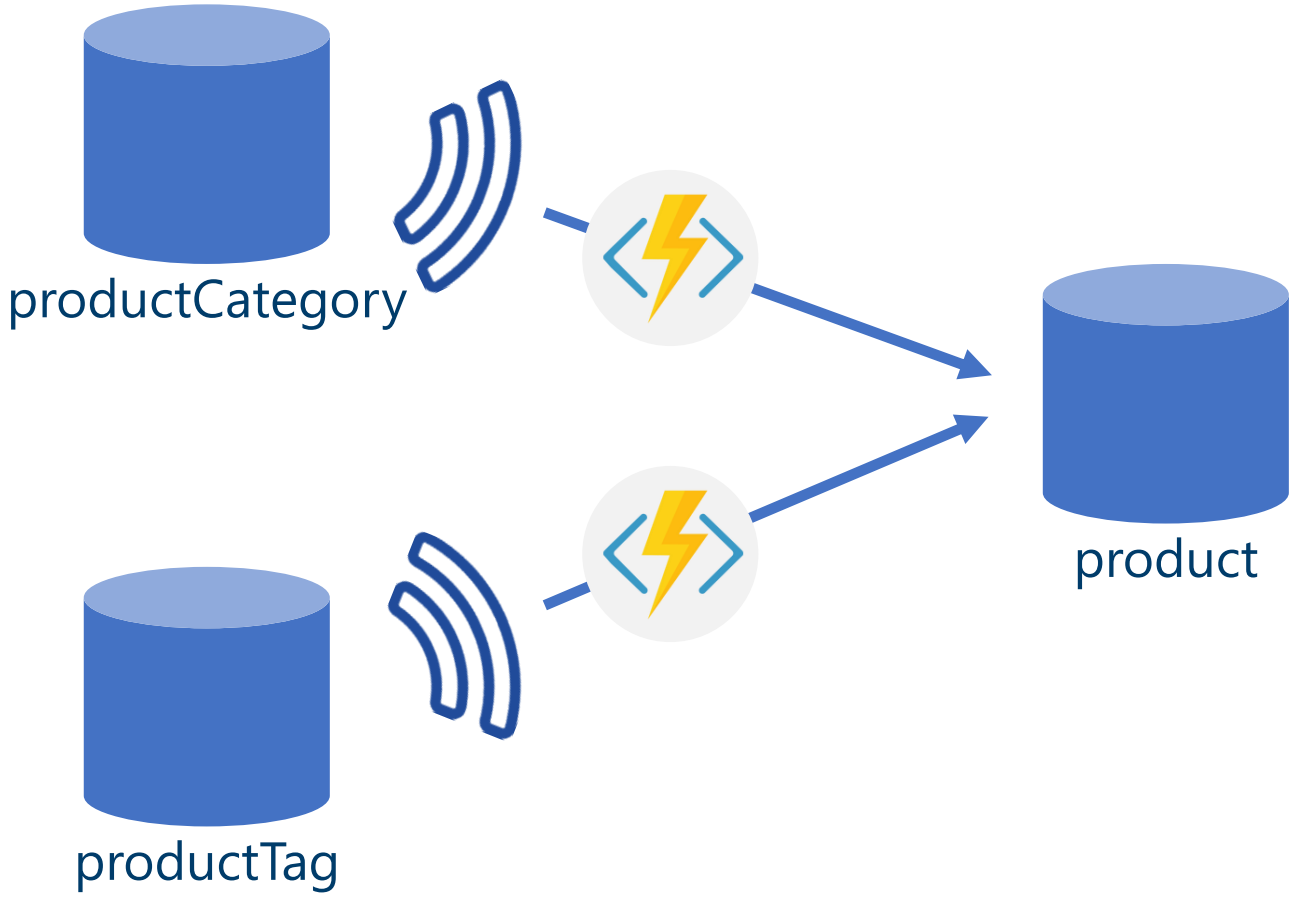
Products: keeping everything in sync



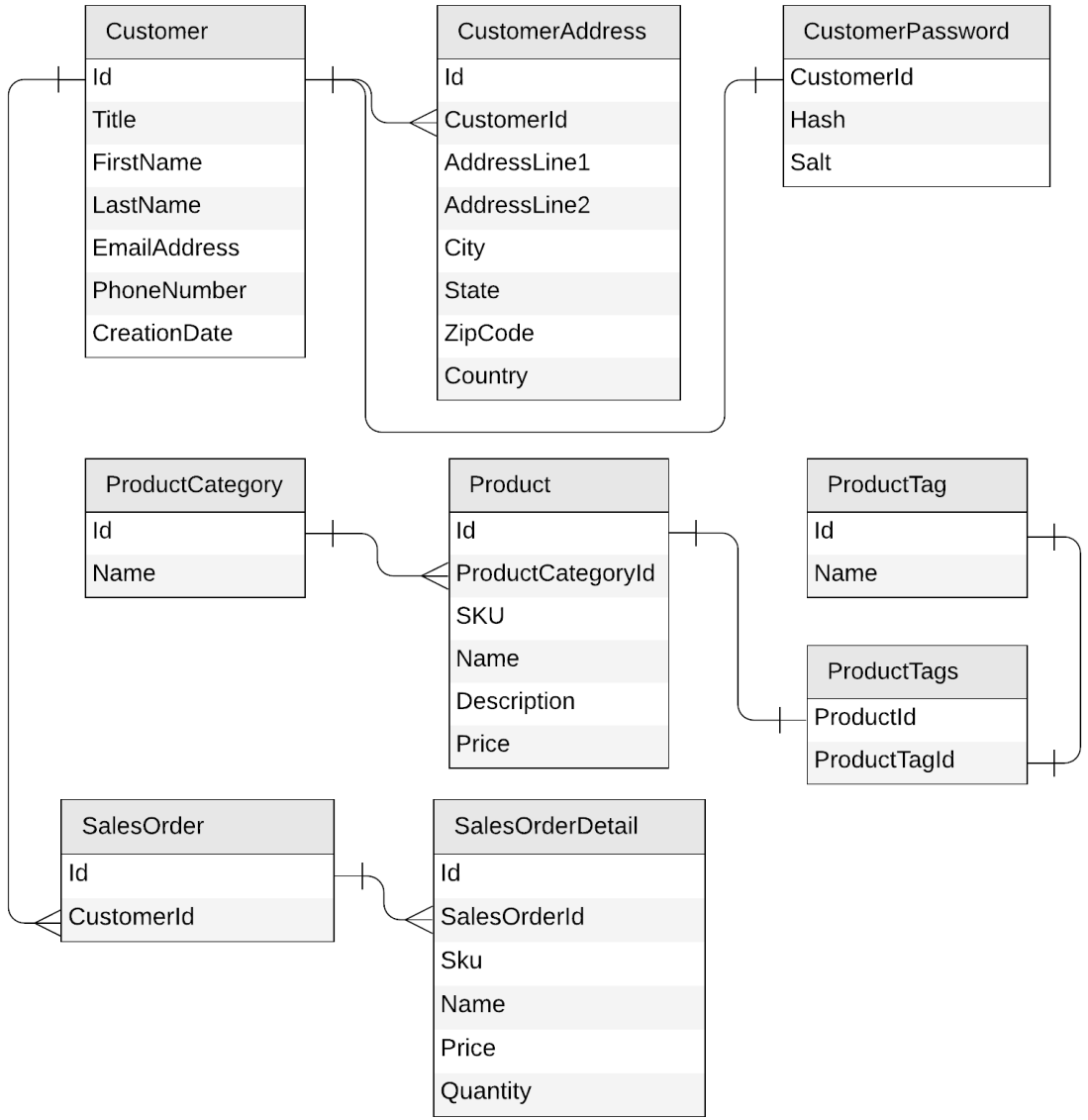
Cosmos DB's change feed

- The change feed is an API exposed by Cosmos DB containers
- It streams any change happening in a container (document creation or update)

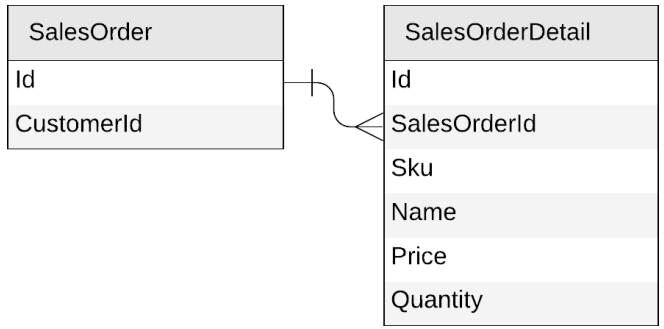
Products: keeping everything in sync



Next: sales orders



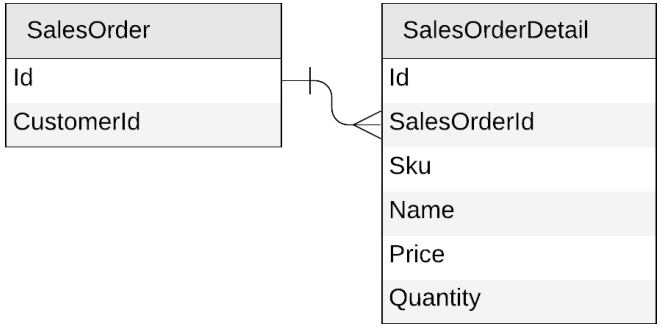
Sales orders




```
{
  "id": "...",
  "customerId": "..."
}
```

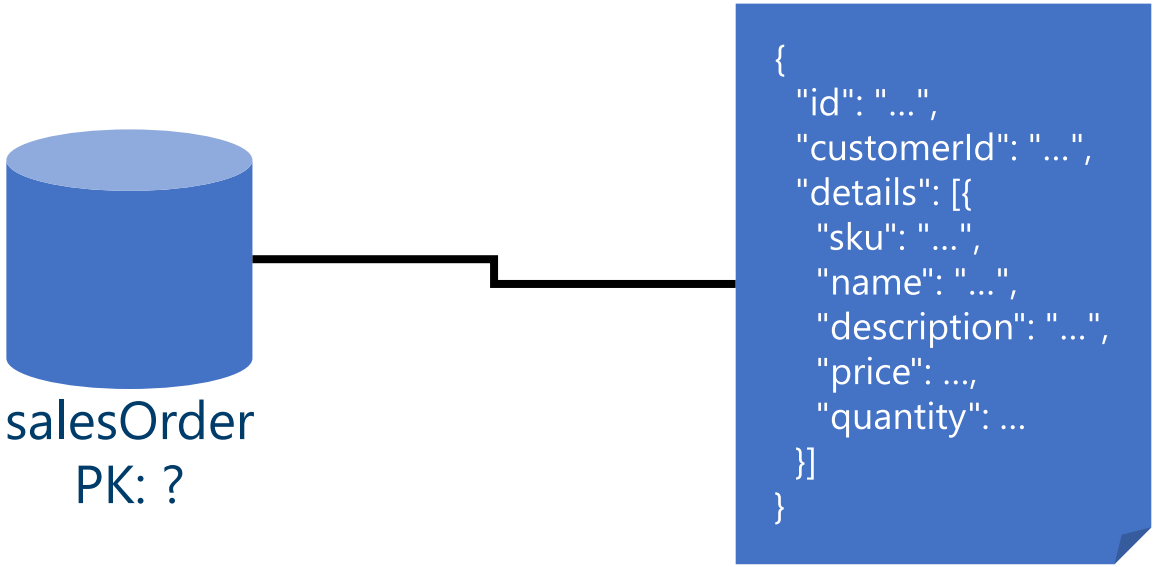
```
{
  "id": "...",
  "salesOrderId": "...",
  "sku": "...",
  "name": "...",
  "description": "...",
  "price": ...,
  "quantity": ...
}
```

Sales orders

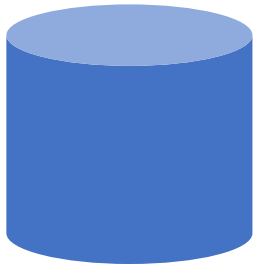



```
{
  "id": "...",
  "customerId": "...",
  "details": [{
    "sku": "...",
    "name": "...",
    "description": "...",
    "price": ...,
    "quantity": ...
  }]
}
```

Sales orders

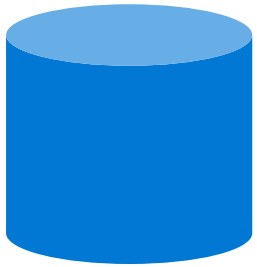


Sales orders



- Create a sales order
- List all sales orders for a customer
- Query top 10 customers by number of sales orders

Sales orders



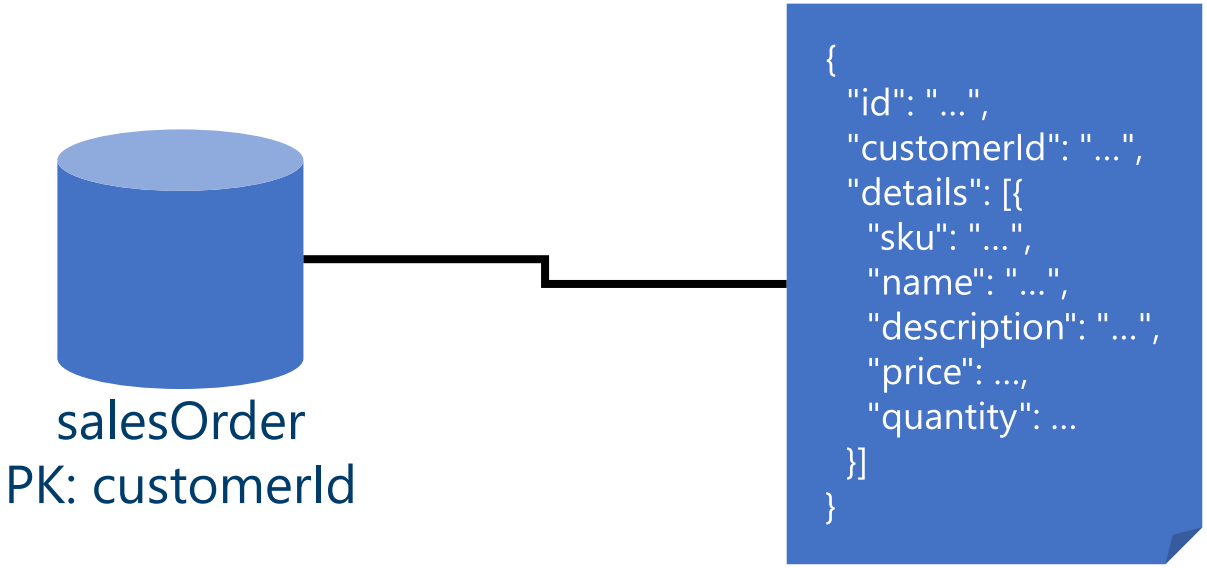
salesOrders
PK: ?

- Create a sales order
- List all sales order for a customer

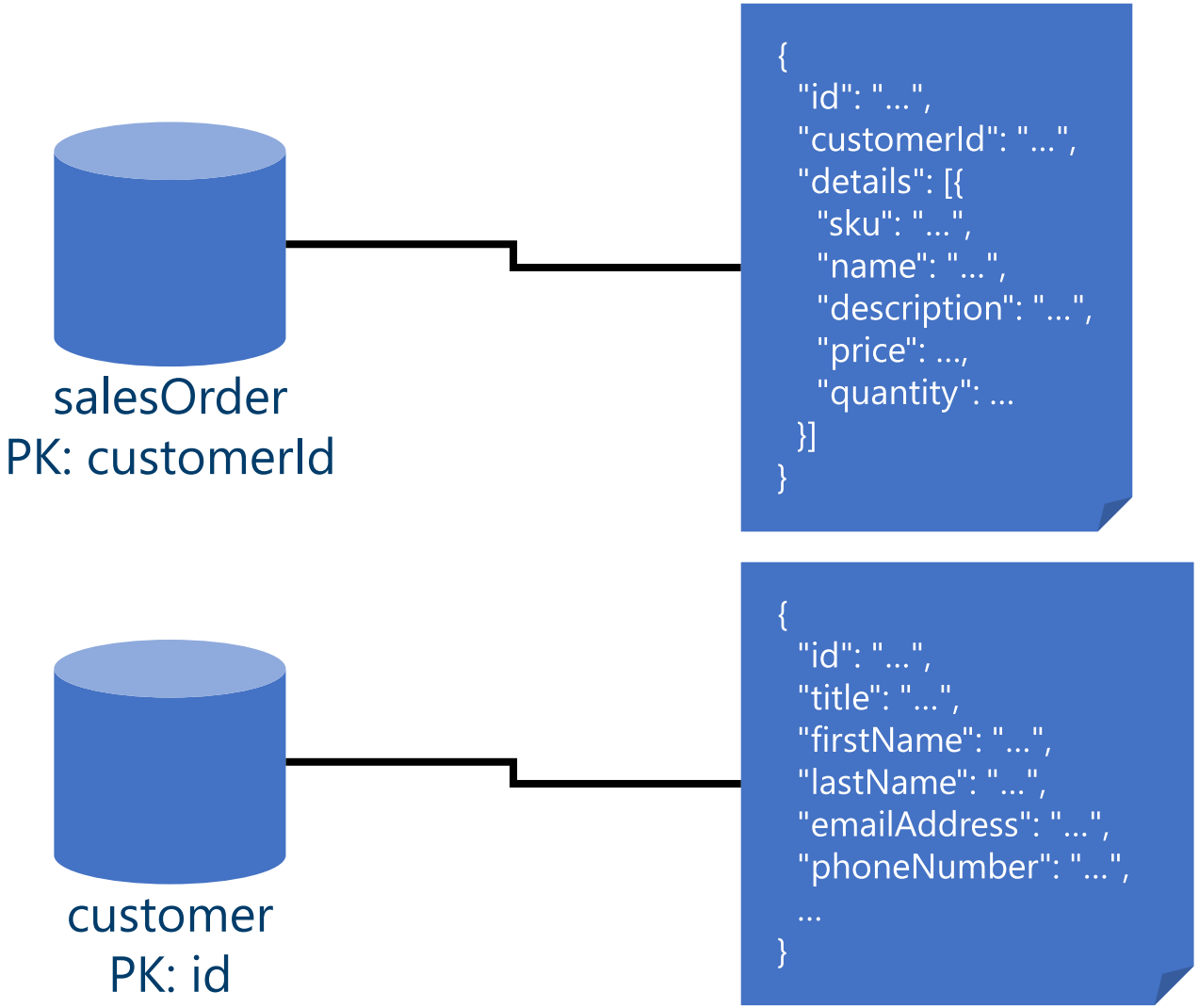
```
SELECT * FROM c WHERE c.customerId = 'CustomerA'
```

CustomerA CustomerB CustomerC

Sales orders



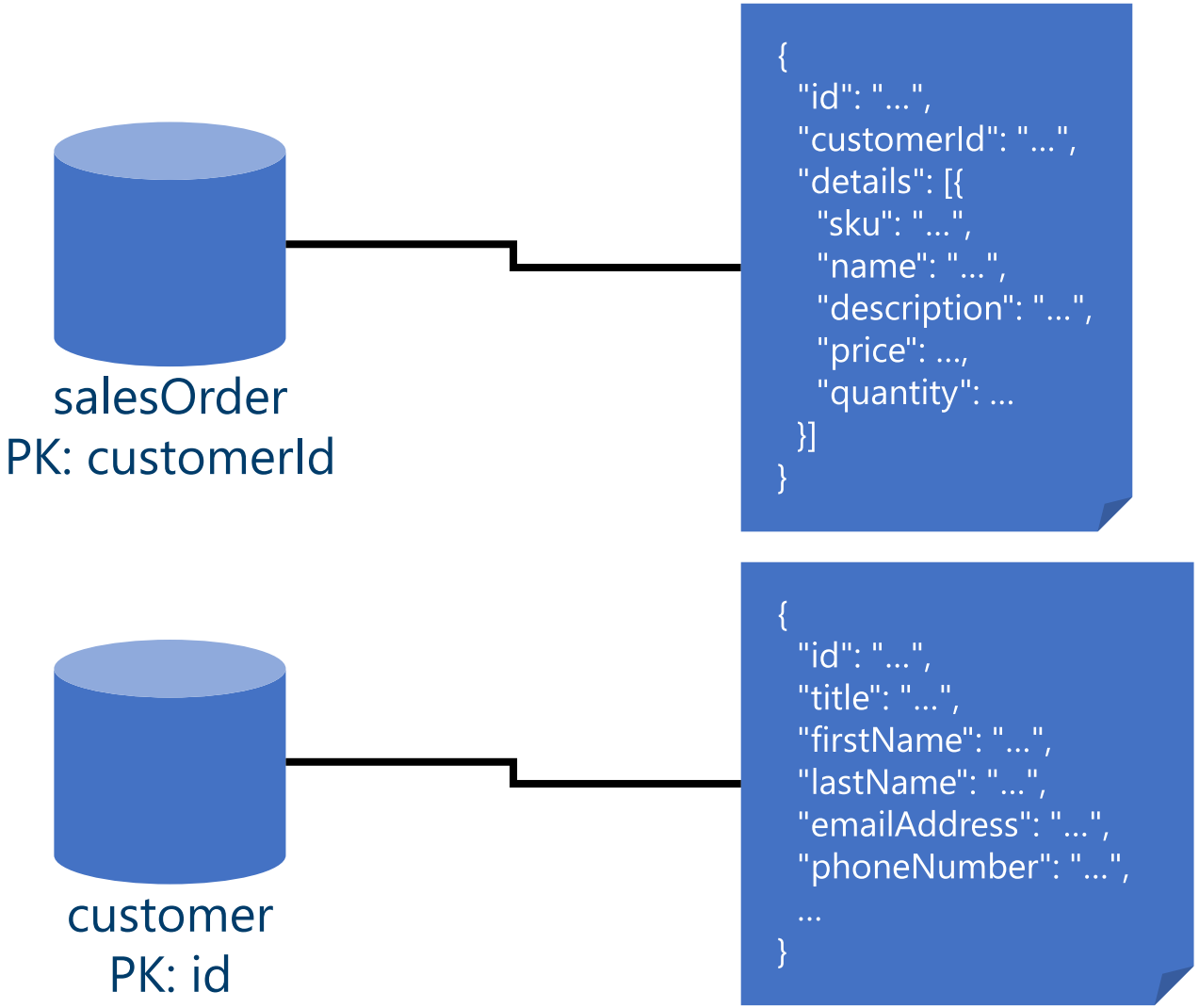
Sales orders



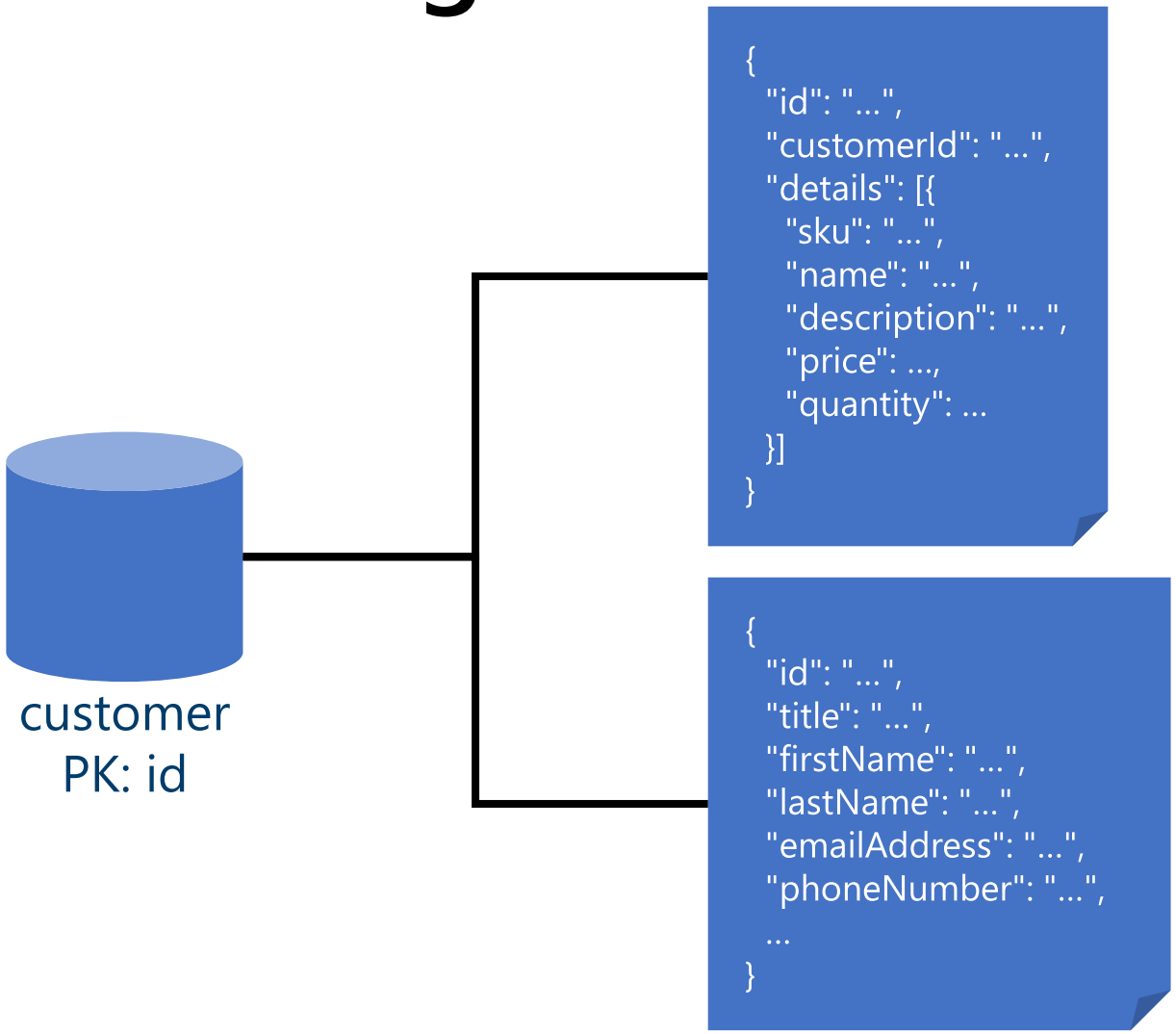
Mixing entities in the same container?

- YES!
- Leverage the schema-agnostic nature of Cosmos DB
- Suitable when different entity types
  - share similar access patterns
  - share the same partition key

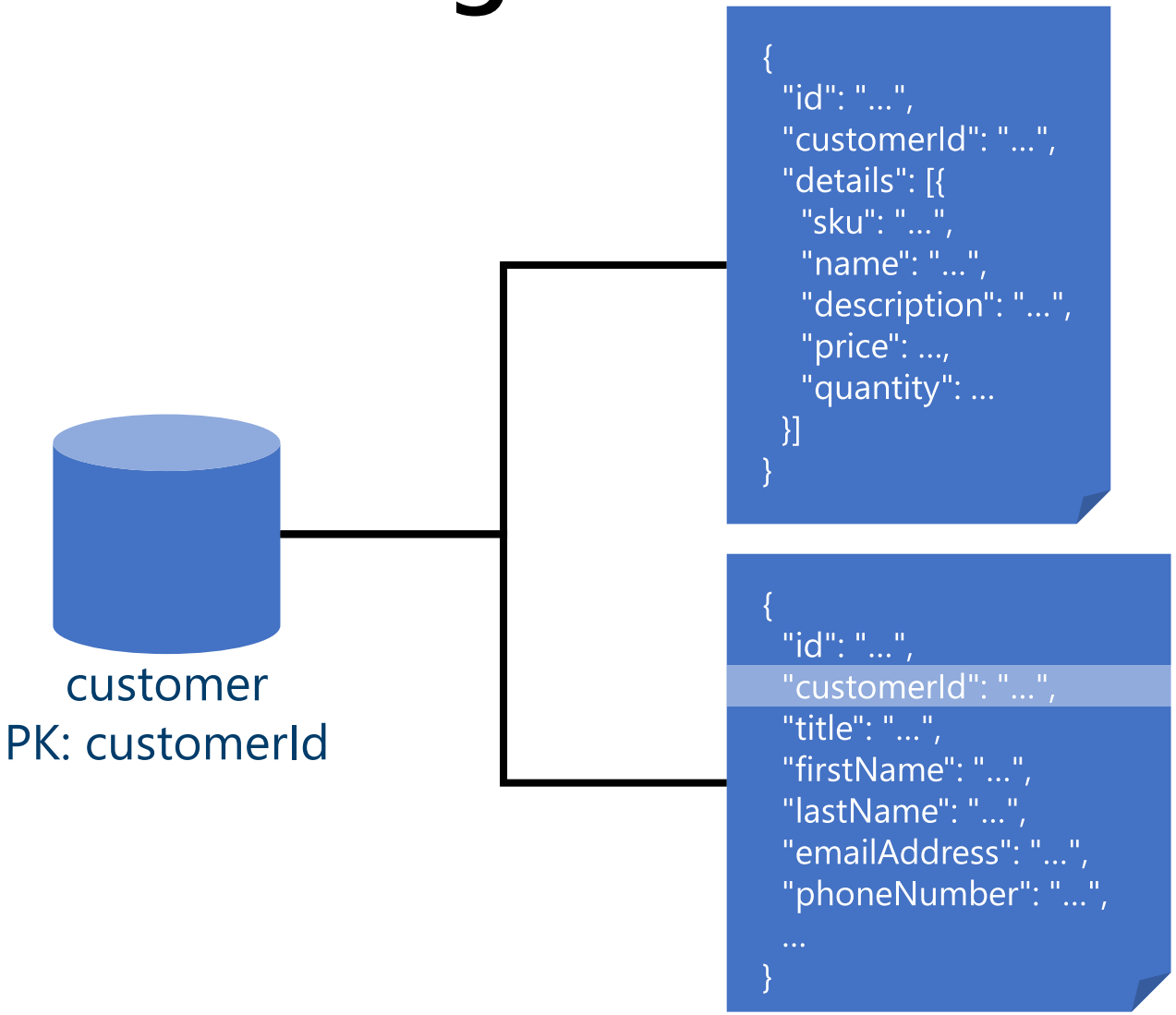
Sales orders



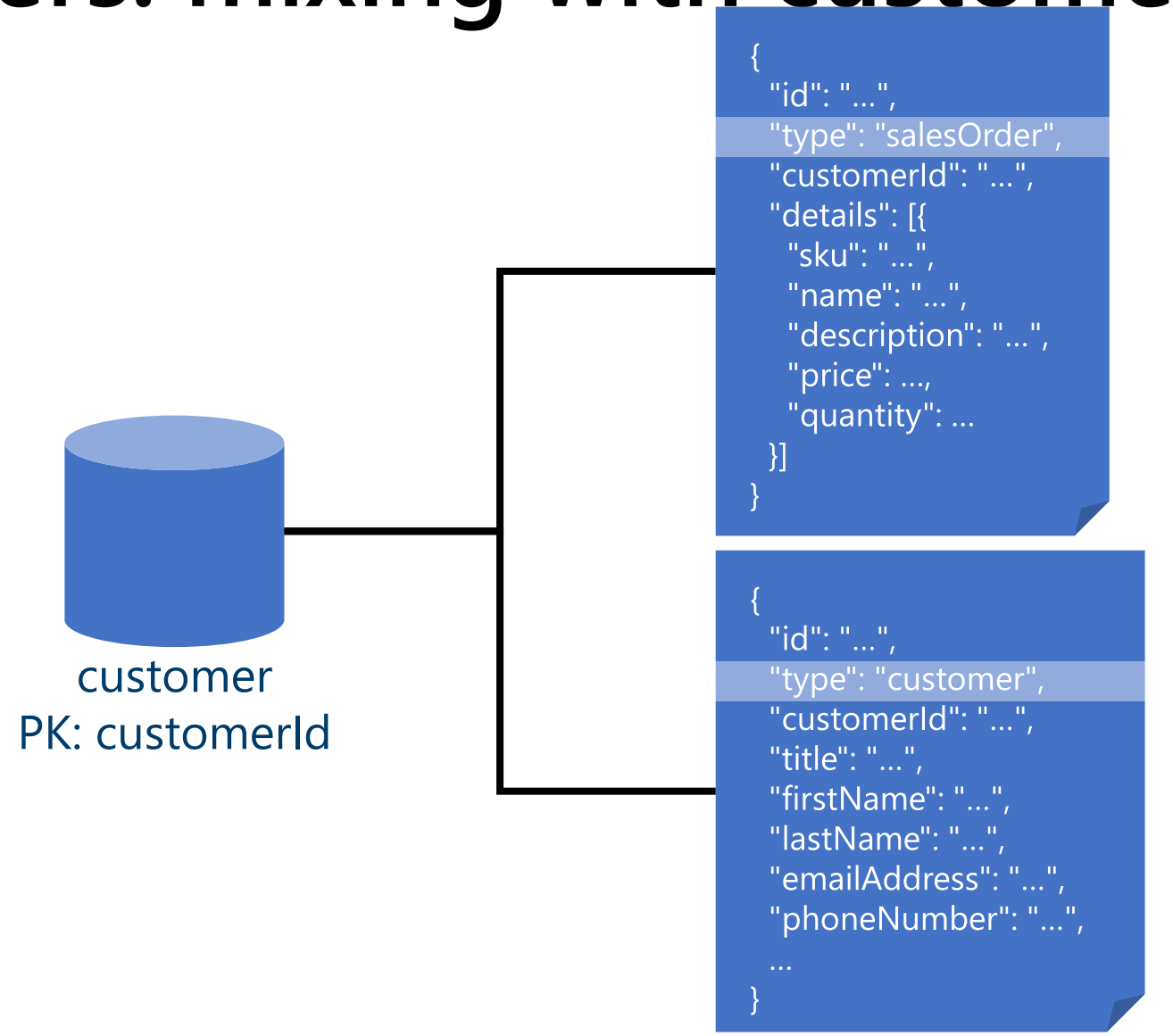
Sales orders: mixing with customers



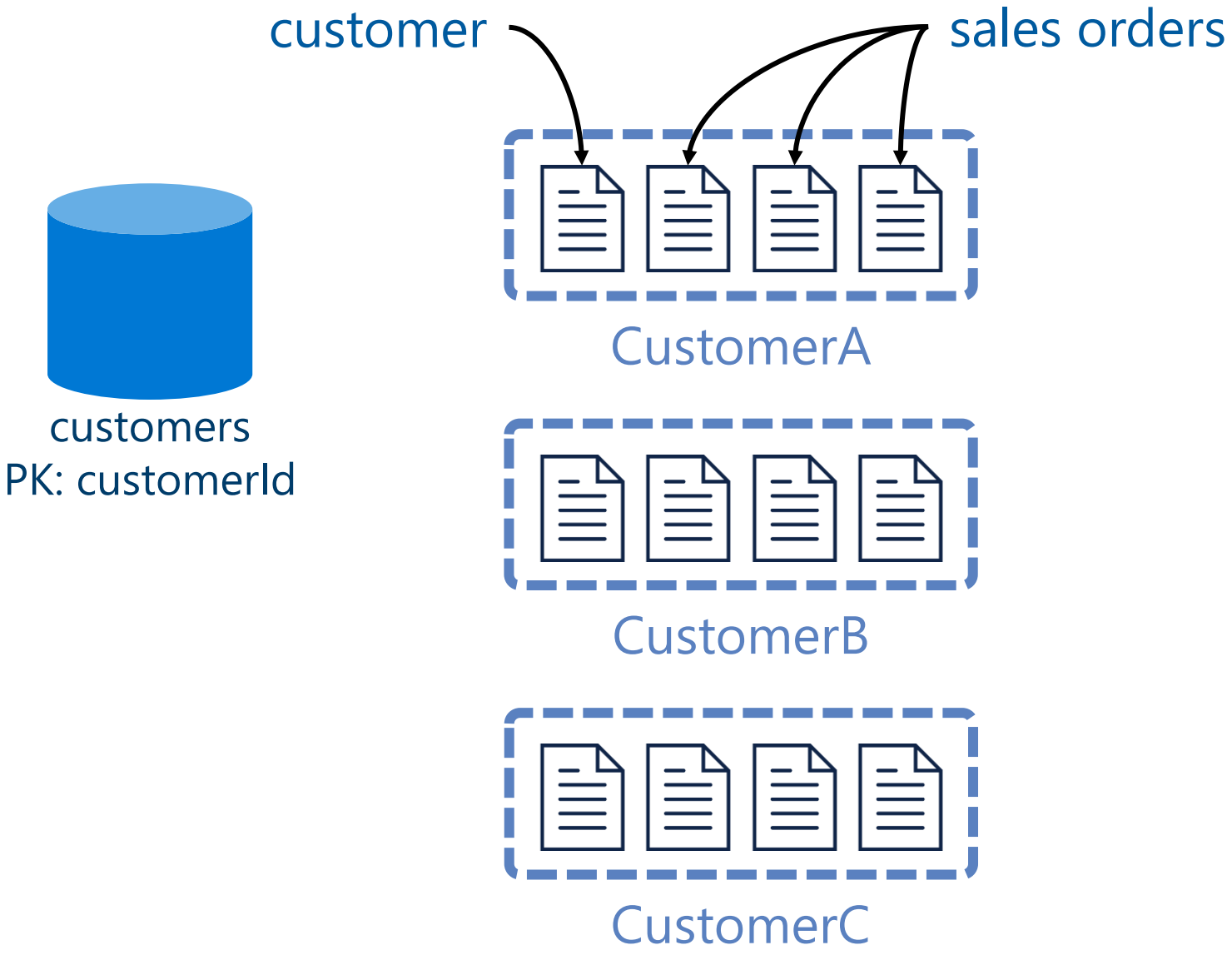
Sales orders: mixing with customers



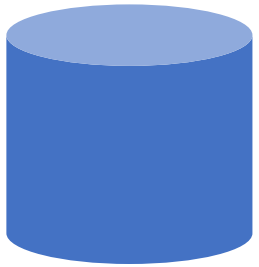
Sales orders: mixing with customers



Sales orders: mixing with customers



Sales orders

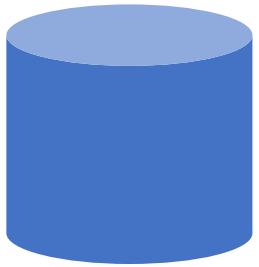


customer
PK: customerId

- Create a sales order
- List all sales order for a customer

```
SELECT * FROM c WHERE c.customerId = 'CustomerA'
AND c.type = 'salesOrder'
```

Sales orders



customer
PK: customerId

- Create a sales order
- List all sales orders for a customer
- Query top 10 customers by number of sales orders

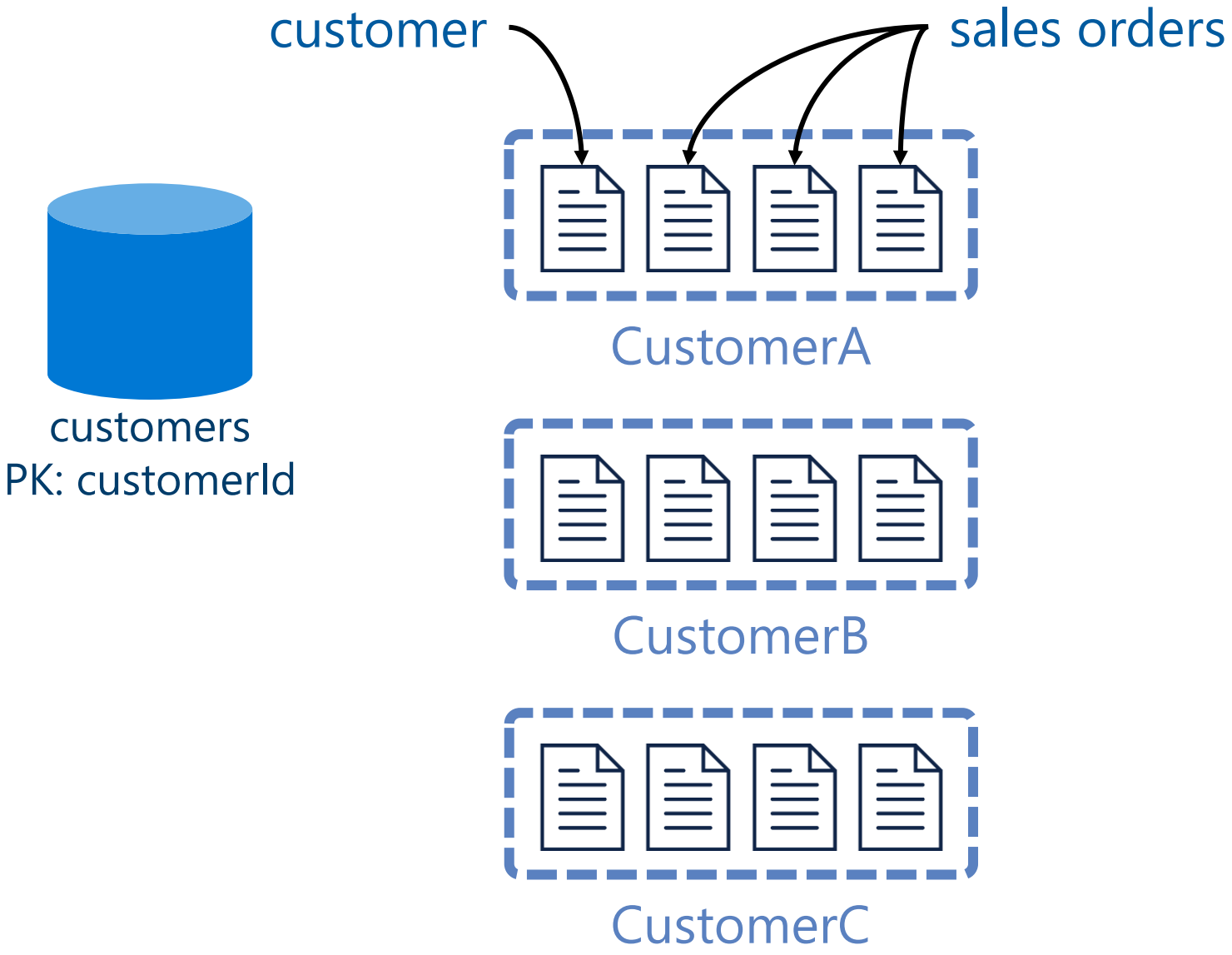
Denormalizing the count of sales orders

```
{
  "id": "...",
  "type": "customer",
  "customerId": "...",
  "title": "...",
  "firstName": "...",
  "lastName": "...",
  "emailAddress": "...",
  "phoneNumber": "...",
  "creationDate": "...",
  "addresses": [{
    "addressLine1": "...",
    "addressLine2": "...",
    ...
  }],
  "password": {
    "hash": "...",
    "salt": "..."
  }
}
```

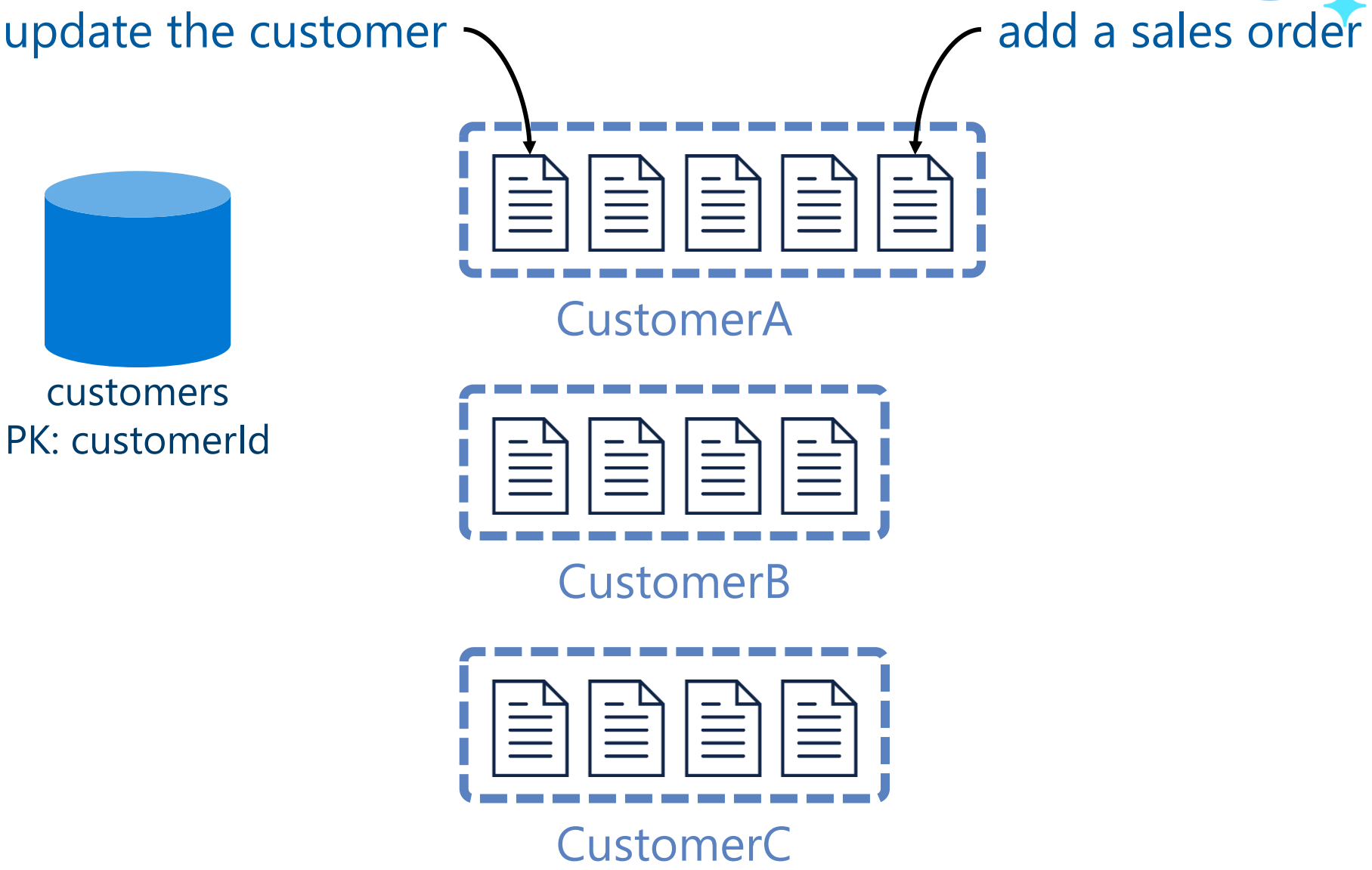
Denormalizing the count of sales orders

```
{
  "id": "...",
  "type": "customer",
  "customerId": "...",
  "title": "...",
  "firstName": "...",
  "lastName": "...",
  "emailAddress": "...",
  "phoneNumber": "...",
  "creationDate": "...",
  "addresses": [{
    "addressLine1": "...",
    "addressLine2": "...",
    ...
  }],
  "password": {
    "hash": "...",
    "salt": "..."
  },
  "salesOrderCount": ...
}
```

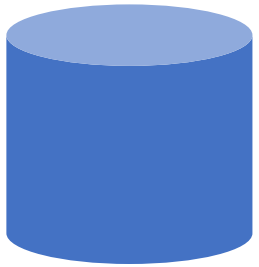
Denormalizing the count of sales orders



Denormalizing the count of sales orders



Sales orders

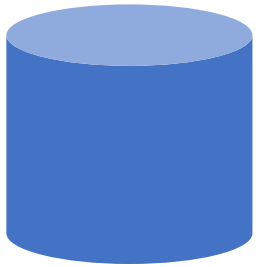


customer
PK: customerId

- Create a sales order
- List all sales order for a customer
- Query top 10 customers by number of sales orders

```
SELECT TOP 10 * FROM c WHERE c.type = 'customer'
ORDER BY c.salesOrderCount DESC
```

Sales orders

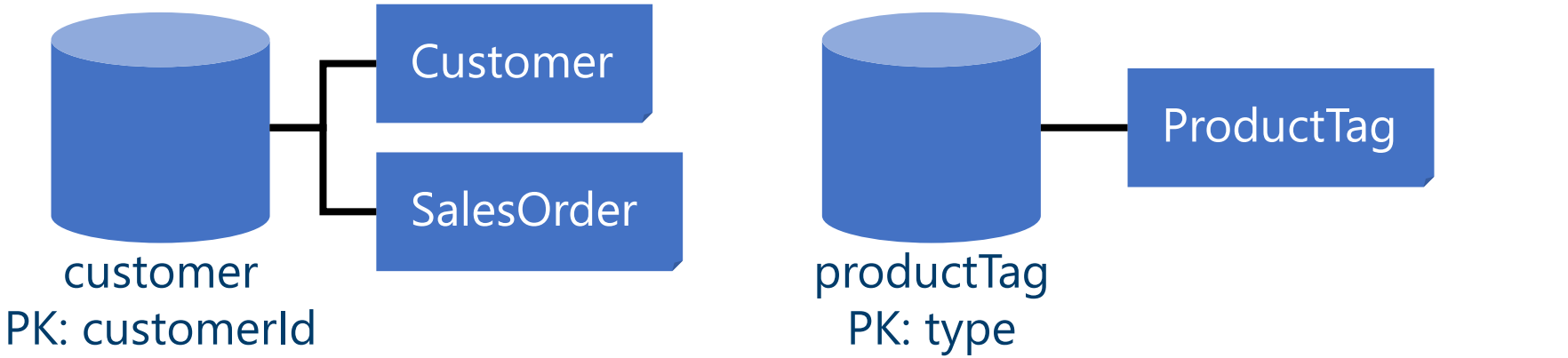


customer
PK: customerId

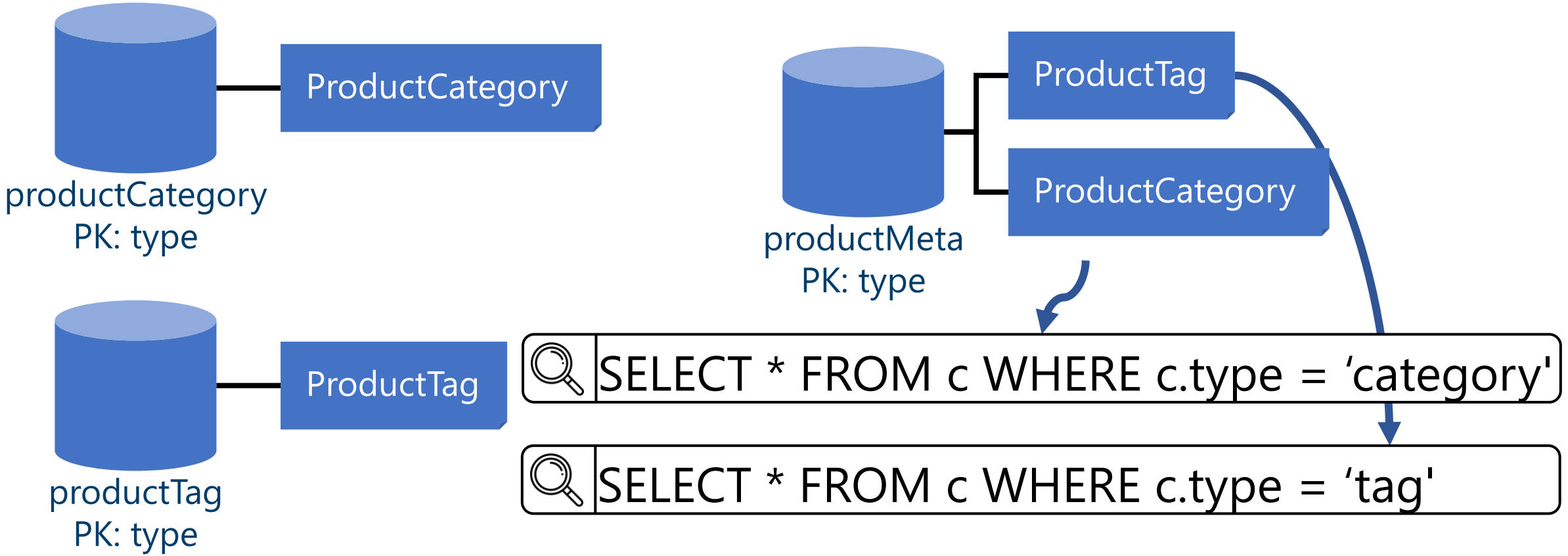
- Create a sales order
- List all sales order for a customer
- List customers by descending number of sales orders

```
SELECT TOP 10 * FROM c WHERE c.type = 'customer'
ORDER BY c.salesOrderCount DESC
```

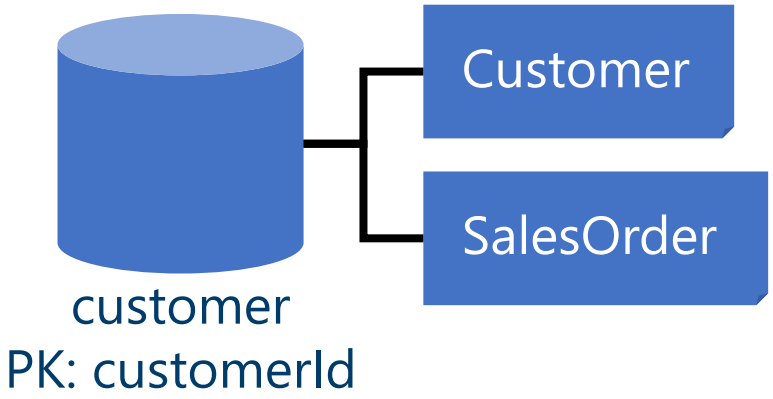
Our final design

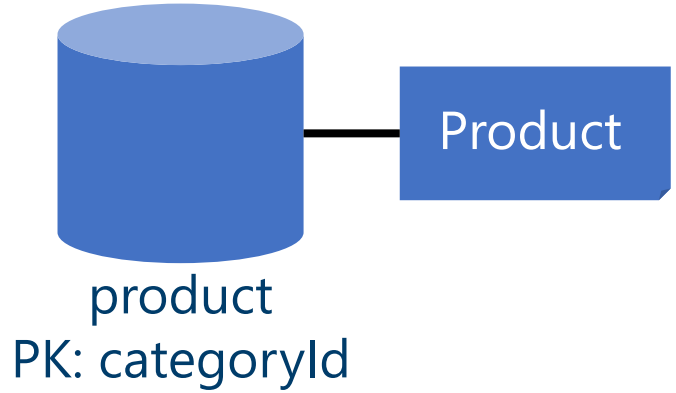


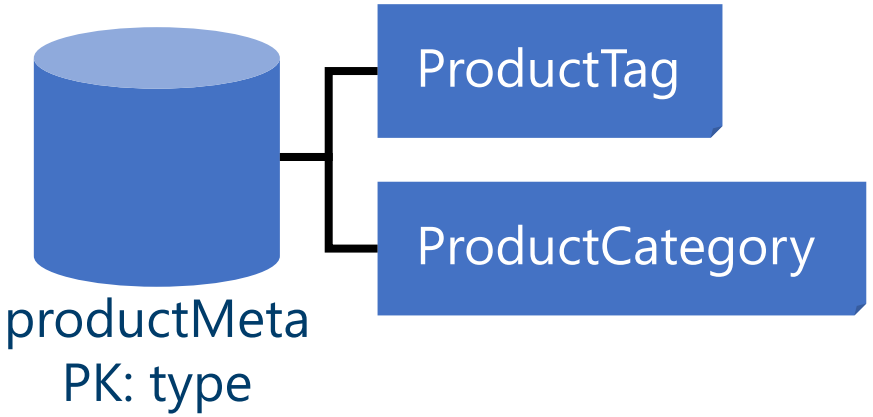
Our final design



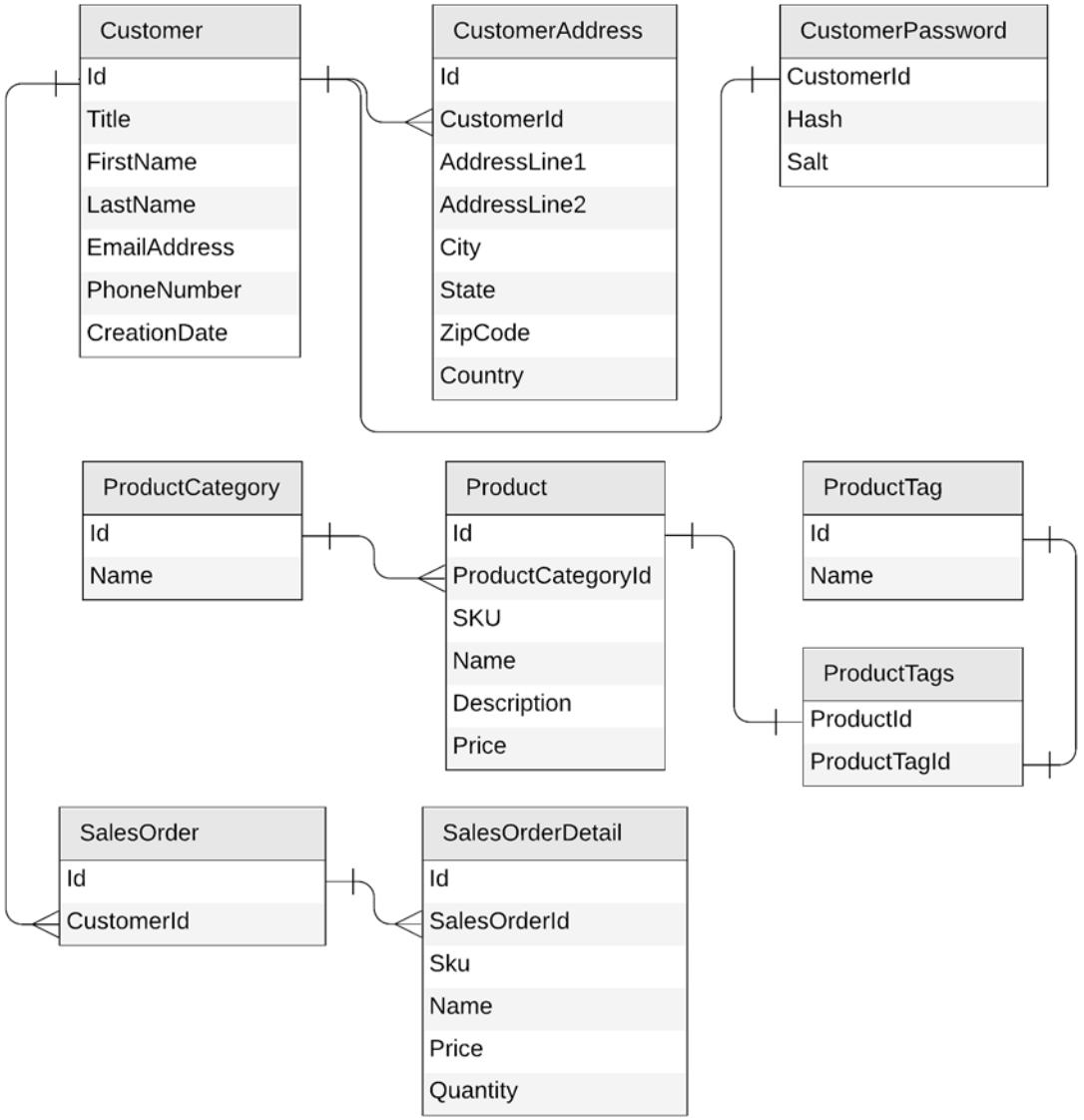
Our final design, optimized!







Original Relational Model



Key takeaways

- Identify key access patterns and design for them
- Partitioning is critical for best performance and scalability
- Materialize relations between entities by:
  - Embedding
  - Denormalizing and pre-aggregating
  - Storing related entities in the same container
- Different methods to implement denormalization
  - Using the change feed
  - Using stored procedures or transactional batch

Public Preview: Relational Database to Azure Cosmos DB Migration Assistant

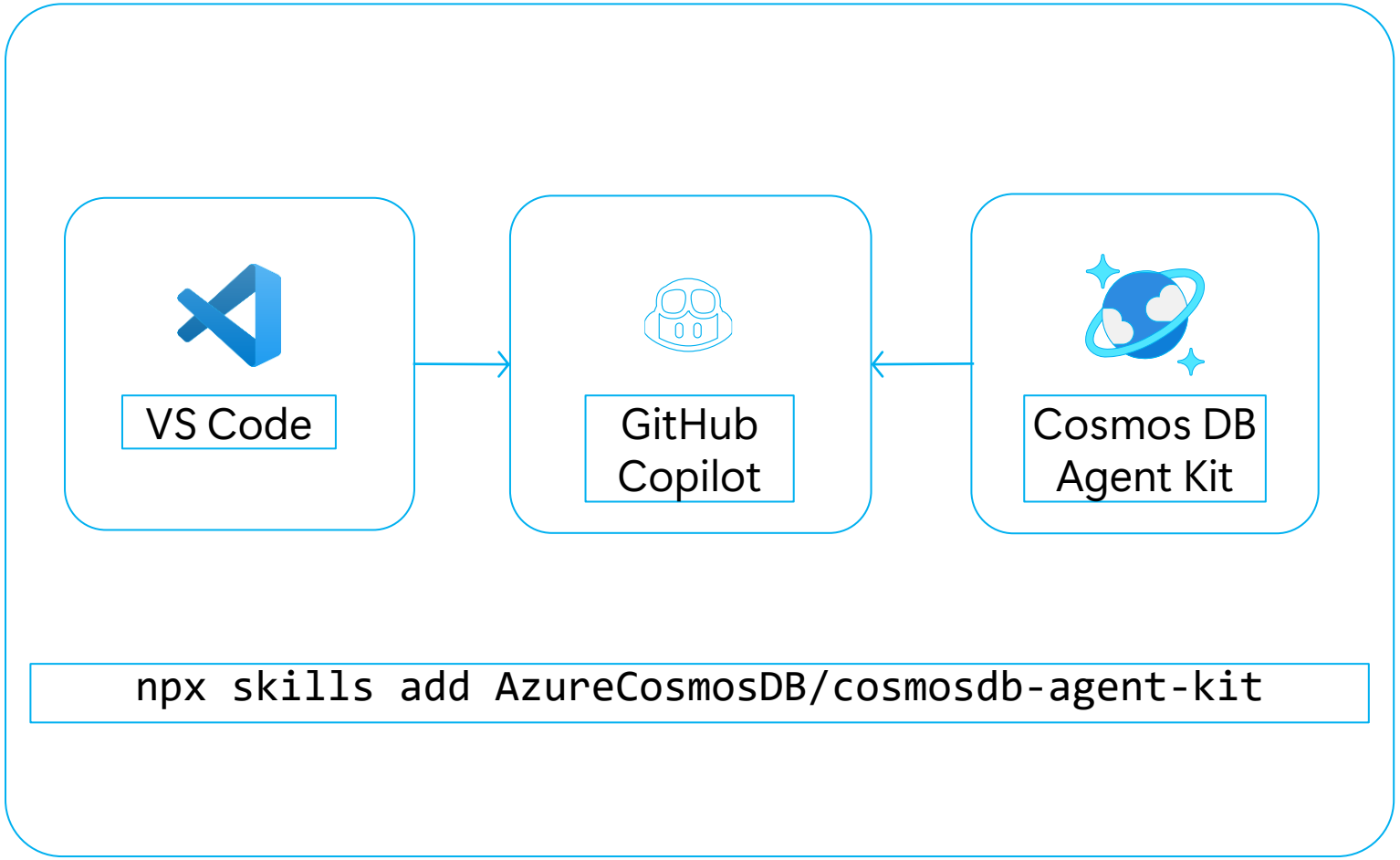
Accelerate migration from relational databases to Azure Cosmos DB

- **AI-assisted migration workflow** – Guides discovery, assessment, schema conversion, and provisioning
- **Supports major relational databases** – Oracle, SQL Server, PostgreSQL, and MySQL
- **Generate NoSQL-ready designs** – Creates optimized Azure Cosmos DB containers and data models
- **Assist with application modernization** – Helps convert application code for Azure Cosmos DB
- **Integrated developer experience** – Manage migration artifacts directly in Visual Studio Code

Announcing Public Preview at Build 2026

Azure Cosmos DB Migration Assistant for RDBMS to NoSQL

Azure Cosmos DB Agent Kit



Skills

- Data Modeling
- Partition Keys
- Queries
- SDK Best Practices
- Indexing
- Throughput
- High Availability
- Monitoring

https://aka.ms/azurecosmosdb-agent-kit

Azure Cosmos DB Agent Kit Overview

- Azure Cosmos DB Agent Kit is an open-source collection of skills that extends AI coding assistants with Azure Cosmos DB best practices. Built on the Agent Skills format, it works with GitHub Copilot, Claude Code, Gemini CLI, etc.
- The kit helps developers optimize:
  - Partitioning
  - Queries
  - data modeling
  - Scalability
  - Reliability
- Using curated production guidance, code review patterns, and recommendations for Cosmos DB applications.

Stop Burning RUs: Live AI App Code Review with the Cosmos DB Agent Kit | Azure Cosmos DB Conf 2026

Going further

- Demos and links to sample data.
  - github.com/AzureCosmosDB/CosmicWorks
  - github.com/AzureCosmosDB/CosmicWorksJava
- New Microsoft Learn Labs
  - https://aka.ms/msl-modeling-partitioning-1
  - https://aka.ms/msl-modeling-partitioning-2
- Awesome videos
  - youtube.com/azurecosmosdb
- Monthly User Group
  - meetup.com/azure-cosmos-db-global-user-group
- Weekly Live Podcast
  - aka.ms/CosmosDBLiveTV